

Hermes Agent Advance Handbook

Hermes Agent Advance Handbook — Achievement Mastery

Scope: The complete Hermes Achievements system — condensed from the 500-line deep reference manual into a single study handbook.

Source: `docs/hermes-achievements-training.md` (517 lines, 25 KB)

Format: Handbook — not unit-based. Organized by category with exact thresholds, strategies, and quick commands.

Hermes Version: v0.15.1 — Bundled plugin (no separate install)

Date: June 2026

Table of Contents

1. System Architecture
2. The 9 Categories — Full Catalog
3. Secret Achievements
4. Playstyle Strategies
5. API Quick Reference
6. Training Exercises
7. Mastery Roadmap
8. Advance Track Units

1. System Architecture

What It Is

Hermes Achievements is a **Steam-style gamification layer** built into the Hermes Dashboard. It scans your real local session history (`~/.hermes/state.db`) and awards 60+ collectible tiered badges across 9 behavioral categories.

Why: Motivation, meta-learning, benchmarking, community.

When: After every session save and dashboard rescan.

Tier Progression

Copper → Silver → Gold → Diamond → Olympian
Easy Moderate Hard Expert Mastery

Every badge has 5 thresholds. Progress tracks cumulatively (`lifetime`) or by peak (`best_session`) or by multi-condition (`multi_condition`).

Achievement States

State	Icon	Meaning	Visibility
-------	------	---------	------------

Unlocked		Earned at least one tier	Always
Discovered		Known, progress visible	After first signal
Secret		Hidden until triggered	Invisible

Scanning Engine

```
~/hermes/state.db (SQLite session history)
  ↓
First scan: incremental, background
  ↓
Snapshot cache → fast warm loads (120s TTL)
  ↓
Checkpoint reuse for unchanged sessions
  ↓
state.json (persistent unlocks + timestamps)
```

Performance: First scan = background, non-blocking. Rescans = sub-second cached.

Where Files Live

```
~/hermes/plugins/hermes-achievements/
├─ dashboard/
│  └─ plugin_api.py      ← Flask routes, badge definitions
├─ state.json            ← YOUR progress (never overwritten on update)
├─ scan_snapshot.json    ← Cached scan payload
└─ scan_checkpoint.json  ← Per-session stats hash
```

Critical: `state.json` is your persistent unlock state. Git pulls, plugin updates, and Hermes upgrades **never** touch it. New versions only evaluate new badges against existing history.

2. The 9 Categories — Full Catalog

2.1 Usage Milestones (16 badges)

Lifetime counters. These are the backbone of your Legend run.

ID	Name	Kind	Metric	Copper	Silver	Gold	Diamond	Olympian
first_blood	First Blood	lifetime	First session	1	—	—	—	—
centurion	Centurion	lifetime	Total sessions	100	300	1K	3K	10K
marathon_runner	Marathon Runner	best_session	Longest session (min)	60	180	480	1,200	3,000
tool_century	Tool Century	lifetime	Total tool calls	10K	30K	100K	300K	1M
file_master	File Master	lifetime	File operations	5K	15K	50K	150K	500K
code_executor	Code Executor	lifetime	execute_code runs	1K	3K	10K	30K	100K

terminal_tyrant	Terminal Tyrant	lifetime	Terminal commands	2K	6K	20K	60K	200K
web_wanderer	Web Wanderer	lifetime	Web search/extract	5K	15K	50K	150K	500K
vision_seeker	Vision Seeker	lifetime	Vision/image analysis	100	300	1K	3K	10K
tts_bard	TTS Bard	lifetime	Text-to-speech	50	150	500	1,500	5K
image_gen_artisan	Image Gen Artisan	lifetime	Image generations	50	150	500	1,500	5K
compression_survivor	Compression Survivor	lifetime	Context compressions	10	30	100	300	1K
checkpoint_champion	Checkpoint Champion	lifetime	Checkpoints created	50	150	500	1,500	5K
memory_packer	Memory Packer	lifetime	Memory consolidations	10	30	100	300	1K
curator_caller	Curator Caller	lifetime	Curator invocations	5	15	50	150	500
legend	Legend	multi_condition	ALL metrics at Olympian	—	—	—	—	—

Key insight: `tool_century` and `terminal_tyrant` will unlock naturally with daily use. `legend` requires reaching Olympian in every other category — the ultimate endgame.

2.2 Tool Proficiency (17 badges)

Tool usage, autonomy, and the debugging that comes with it.

Autonomy sub-group (7):

ID	Name	Kind	Metric	C	S	G	D	O
let_him_cook	Let Him Cook	best_session	Max tool calls in one session	200	500	1,200	3,000	8,000
autonomous_avalanche	Autonomous Avalanche	lifetime	Total tool calls	1K	3K	8K	20K	50K
toolchain_maxxer	Toolchain Maxxer	best_session	Max distinct tools in one session	18	28	45	70	100
full_send	Full Send	multi_condition	Terminal $\geq$ 180 + File $\geq$ 120 + Web $\geq$ 60	—	—	—	—	—
subagent_commander	Subagent Commander	lifetime	<code>delegate_task</code> calls	5	40	100	1K	5K
background_process_enjoyer	Background Process Enjoyer	lifetime	Background process calls	300	800	2K	6K	15K
cron_necromancer	Cron Necromancer	lifetime	Cron/scheduled jobs	1K	3K	8K	20K	50K

Debugging sub-group (10):

ID	Name	Kind	Metric	C	S	G	D	O	Secret
red_text_connoisseur	Red Text Connoisseur	lifetime	Total errors	1.5K	4K	10K	25K	75K	
stack_trace_sommelier	Stack Trace Sommelier	lifetime	Traceback events	300	1K	3K	8K	20K	
actually_read_the_logs	Actually Read The Logs	lifetime	Log reads	1K	3K	8K	20K	50K	
port_3000_taken	Port 3000 Is Taken	lifetime	Port conflicts	15	40	100	300	1K	✓
permission_denied_any_percent	Permission Denied Any%	lifetime	Permission errors	25	75	200	600	1.5K	✓
dependency_hell_tourist	Dependency Hell Tourist	multi_condition	Install errors≥25 + successes≥10	—	—	—	—	—	
the_fix_was_restarting	The Fix Was Restarting	multi_condition	Restart-after-error≥50 + total errors≥4K	—	—	—	—	—	
forgot_the_env_var	Forgot The Env Var	lifetime	Env var errors	5K	15K	40K	100K	250K	✓
yaml_colon_incident	YAML Colon Incident	lifetime	YAML errors	1K	3K	8K	20K	50K	✓
docker_name_collision	Docker Name Collision	lifetime	Docker conflicts	75	200	600	1.5K	4K	✓

Strategy: `subagent_commander` + `background_process_enjoyer` unlock together. `let_him_cook` requires one massive session — use `delegate_task` for parallelism on a 2015 MBP with thermal throttling.

2.3 Model Diversity (5 badges)

New in v0.2.x. Celebrates model/provider exploration.

ID	Name	Kind	Metric	C	S	G	D	O
five_model_flight	Five-Model Flight	best_session	5+ models in one session	—	—	—	—	—
provider_polyglot	Provider Polyglot	lifetime	Distinct providers used	3	5	8	12	18
claude_confidant	Claude Confidant	lifetime	Claude-specific sessions	50	150	400	1K	3K
gemini_cartographer	Gemini Cartographer	lifetime	Gemini-specific sessions	50	150	400	1K	3K
open_weights_pilgrim	Open Weights Pilgrim	lifetime	Local/OSS model sessions	50	150	400	1K	3K

Strategy: Use `/model` to switch providers mid-session for `five_model_flight`. `provider_polyglot` requires trying Ollama Cloud (Kimi-K2.6, GLM-5.1, Qwen), Anthropic, OpenRouter, and local models.

2.4 Memory & Research (7 badges)

ID	Name	Kind	Metric	C	S	G	D	O
<code>rabbit_hole_certified</code>	Rabbit Hole Certified	<code>lifetime</code>	Total web calls	400	1.2K	3K	8K	20K
<code>citation_goblin</code>	Citation Goblin	<code>lifetime</code>	<code>web_extract</code> calls	100	300	1K	3K	8K
<code>docs_archaeologist</code>	Docs Archaeologist	<code>lifetime</code>	Documentation reads	200	600	1.5K	4K	10K
<code>search_orchestrator</code>	Search Orchestrator	<code>best_session</code>	Max web calls in one session	50	150	400	1K	3K
<code>memory_keeper</code>	Memory Keeper	<code>lifetime</code>	Memory events	100	300	1K	3K	8K
<code>memory_palace</code>	Memory Palace	<code>lifetime</code>	Memory write events	100	300	1K	3K	8K
<code>context_dragon</code>	Context Dragon	<code>lifetime</code>	Context events	5K	15K	40K	100K	250K

Strategy: `rabbit_hole_certified` is the signature research badge. Combine `web_search` + `web_extract` + `browser` for deep dives. `memory_keeper` + `memory_palace` unlock together with regular `memory` tool use.

2.5 Skills & Automation (8 badges)

ID	Name	Kind	Metric	C	S	G	D	O
<code>skillsmith</code>	Skillsmith	<code>lifetime</code>	Skill events	5K	15K	40K	100K	250K
<code>skill_issue_skill_created</code>	Skill Issue? Skill Created.	<code>lifetime</code>	<code>skill_manage</code> events	25	75	200	600	1.5K
<code>gateway_dweller</code>	Gateway Dweller	<code>lifetime</code>	Gateway events	5K	15K	40K	100K	250K
<code>plugin_goblin</code>	Plugin Goblin	<code>lifetime</code>	Plugin events	1K	3K	8K	20K	50K
<code>rollback_wizard</code>	Rollback Wizard	<code>lifetime</code>	Rollback events	500	1.5K	4K	10K	25K

Note: `memory_keeper`, `memory_palace`, `context_dragon`, `subagent_commander`, `cron_necromancer`, `background_process_enjoyer` badges also appear in Skills & Automation tracking context (overlap is intentional — achievements are not mutually exclusive by category).

Strategy: `skillsmith` is the highest-tier native badge. Use `skill_manage` frequently. Gateway Dweller requires running the messaging gateway (Telegram, Discord, etc.).

2.6 Session Quality (7 badges)

ID	Name	Kind	Metric	C	S	G	D	O
<code>supposed_to_be_quick</code>	This Was Supposed To Be Quick	<code>best_session</code>	Max messages in one session	300	600	1.2K	2.5K	6K
<code>one_more_small_change</code>	One More Small Change	<code>best_session</code>	Max file tool calls in one session	150	400	1K	3K	8K
<code>vibe_architect</code>	Vibe Architect	<code>best_session</code>	Max files touched in one session	300	700	1.5K	4K	10K

<code>weekend_warrior</code>	Weekend Warrior	<code>lifetime</code>	Weekend sessions	10	30	80	200	500
<code>night_owl</code>	Night Owl	<code>lifetime</code>	Night sessions (22:00–05:00)	20	60	150	400	1K
<code>early_bird_cron</code>	Early Bird Cron	<code>lifetime</code>	Morning cron jobs (05:00–09:00)	5	15	40	100	250
<code>marathon_runner</code>	Marathon Runner	<code>best_session</code>	Longest session (min)	60	180	480	1.2K	3K

Strategy: Lifestyle badges are **time-gated** — cannot be rushed. Vibe Coding badges unlock during long debugging/refactoring sessions intentionally.

2.7 Social & Community (3 badges)

ID	Name	Kind	Metric	C	S	G	D	O
<code>shared_session</code>	Shared Session	<code>lifetime</code>	Sessions shared/exported	5	15	40	100	250
<code>team_delegate</code>	Team Delegate	<code>lifetime</code>	Multi-user delegate calls	10	30	80	200	500
<code>cross_platform</code>	Cross Platform	<code>lifetime</code>	Platforms used (CLI + Gateway)	2	3	4	5	6

Strategy: `cross_platform` is the easiest — just connect CLI **and** Telegram gateway. `shared_session` requires exporting trajectories to agentskills.io.

2.8 Secret Achievements (8 badges)

Invisible until your first triggering session. See §3 for unlock hints.

Badge	Unlock Trigger
Port 3000 Is Taken	Server processes hit port conflicts
Permission Denied Any%	Sudo/chmod/Docker permission failures
Forgot The Env Var	Config loading with missing env vars
YAML Colon Incident	YAML edits with syntax errors
Docker Name Collision	Duplicate container names
Rollback Wizard	Frequent <code>/rollback</code> after mistakes
One Character Fix	Fix a bug with a 1-character patch
Legend	Reach Olympian in ALL other badges

Strategy: Most secret badges unlock **accidentally** during normal workflows. Only `legend` requires grind. `rollback_wizard` is actively farmable.

3. Secret Achievements (Detailed)

3.1 How Secret Detection Works

The scanning engine classifies achievements by **regex match** on session tool output:

```
# Conceptual classifier (from plugin_api.py)
HINT_PATTERNS = {
    "port_3000_taken": r"Address already in use.*:3000",
    "permission_denied_any_percent": r"Permission denied|EACCES",
    "forgot_the_env_var": r"KeyError.*env|Environment variable",
    "yaml_colon_incident": r"yaml\. (scanner|parser|constructor)",
    "docker_name_collision": r"container name.*already in use",
    "rollback_wizard": r"/rollback.* executed",
    "one_character_fix": r"single character|one char fix",
}
```

When a pattern matches for the first time:

1. Badge flips from **Secret** (invisible) → **Discovered** (visible, progress=0)
2. All previous session history is re-scanned for additional matches
3. Progress updates to `lifetime count` of matches

3.2 Legend — The Ultimate Badge

Requirement: Reach Olympian tier in **ALL** other 59 badges.

Why it's the endgame: No single strategy works. You need:

- Vibe-coding marathons (`supposed_to_be_quick` + `one_more_small_change` + `vibe_architect`)
- Daily tool grinding (`tool_century`, `terminal_tyrant`, `web_wanderer`)
- Provider polyglot (`provider_polyglot` at 18 providers)
- All model-specific badges at Olympian
- All secret badges maxed out
- All lifestyle badges at Olympian (time-gated — this takes months)

Verification:

```
curl -s http://127.0.0.1:9119/api/plugins/hermes-achievements/achievements \
  | jq '[.achievements[] | select(.tier == "olympian")] | length'
# When this returns 59 or more, Legend is the only badge remaining.
```

4. Playstyle Strategies

Speedrunner — 10 badges in 1 week

Priority	Badge	How
1	First Blood	Complete your first session
2	Red Text Connoisseur	Use Hermes normally — errors happen
3	Actually Read The Logs	Check logs with <code>/logs</code> or <code>read_file</code>
4	Memory Keeper	Use <code>memory</code> tool frequently
5	Rabbit Hole Certified	Research topics with <code>web_search</code>

6	Toolchain Maxxer	Use 18+ different tools in one session
7	Weekend Warrior	Use Hermes on Saturday/Sunday
8	File Master	Edit files with <code>write_file</code> / <code>patch</code>
9	Code Executor	Run Python with <code>execute_code</code>
10	Terminal Tyrant	Execute shell commands

Architect — Maximize Native + Vibe Coding

Priority	Badge	Strategy
1	This Was Supposed To Be Quick	Start a refactor, let it spiral
2	One More Small Change	Iterate on one file 150+ times
3	Vibe Architect	Touch 300+ files in one session
4	Ship First, Ask Later	Git commit + push from Hermes
5	CSS Exorcist	Frontend projects with styling
6	Skillsmith	Create skills from coding patterns
7	Memory Palace	Write memory entries after each session
8	Context Dragon	Long context sessions with compression

Researcher — Information Forager

Priority	Badge	Strategy
1	Rabbit Hole Certified	Deep-dive with 20+ web calls
2	Citation Goblin	<code>web_extract</code> on every source
3	Docs Archaeologist	Read documentation files
4	Search Orchestrator	50+ web calls in one session
5	Provider Polyglot	Switch models with <code>/model</code>
6	Claude Confidant	Use Anthropic Claude exclusively
7	Five-Model Flight	Use 5 models in one long session
8	Open Weights Pilgrim	Run local models via Ollama

Autonomy Engineer — Background Work

Priority	Badge	Strategy
1	Let Him Cook	One session with 200+ tool calls
2	Subagent Commander	Spawn 5+ <code>delegate_task</code> children
3	Background Process Enjoyer	Run <code>terminal(background=True)</code>
4	Cron Necromancer	Schedule jobs with <code>cronjob</code> tool
5	Full Send	Heavy terminal + file + web in one session
6	Autonomous Avalanche	Just keep using Hermes daily

Secret Hunter

Secret	Action
Port 3000 Is Taken	Start a server, then try again
Permission Denied Any%	Run <code>sudo</code> commands that fail
Forgot The Env Var	Load configs with missing env vars
YAML Colon Incident	Edit YAML with syntax errors
Docker Name Collision	<code>docker run --name same twice</code>
Rollback Wizard	Use <code>/rollback</code> after bad edits
One Character Fix	Fix a bug with a 1-char patch
Legend	Olympian everywhere

5. API Quick Reference

Dashboard Endpoints

Base: `http://127.0.0.1:9119/api/plugins/hermes-achievements/`

Method	Endpoint	Returns
GET	<code>/achievements</code>	Full catalog + your progress
GET	<code>/scan-status</code>	Current scan state
GET	<code>/recent-unlocks</code>	Recently earned badges
GET	<code>/sessions/{id}/badges</code>	Badges for a specific session
POST	<code>/rescan</code>	Force full rescan
POST	<code>/reset-state</code>	Wipe all progress (irreversible!)

Useful Commands

```
# Export all progress to JSON
curl -s http://127.0.0.1:9119/api/plugins/hermes-achievements/achievements \
  | jq '.achievements[] | {name, tier, progress, unlocked}' > my_progress.json

# Check recent unlocks
curl -s http://127.0.0.1:9119/api/plugins/hermes-achievements/recent-unlocks \
  | jq '.[0] | {name, tier, unlocked_at}'

# Count Olympian badges
curl -s http://127.0.0.1:9119/api/plugins/hermes-achievements/achievements \
  | jq ' [.achievements[] | select(.tier == "olympian")] | length'

# Force rescan (safe — never wipes progress)
curl -X POST http://127.0.0.1:9119/api/plugins/hermes-achievements/rescan

# Reset state (DANGEROUS — wipes all progress)
curl -X POST http://127.0.0.1:9119/api/plugins/hermes-achievements/reset-state
```

State Files

```
~/hermes/plugins/hermes-achievements/
├─ state.json           # Persistent unlocks + timestamps
├─ scan_snapshot.json   # Cached scan payload (120s TTL)
└─ scan_checkpoint.json # Per-session stats cache
```

6. Training Exercises

Exercise 1: First Blood + Toolchain Maxxer (30 min)

Goal: Unlock 2 badges in one session.

1. Start Hermes: `hermes --tui`
2. Use at least 18 different tools:
 - `web_search`, `web_extract`, `read_file`, `write_file`, `terminal`, `execute_code`, `browser_navigate`, `memory`, `skill_manage`, `delegate_task`, `cronjob`, `todo`, `clarify`, `session_search`, `fact_store`, `image_generate`, `tts`, `vision_analyze`
3. Type `/save` and exit
4. Run: `hermes dashboard` → Achievements tab → verify

Exercise 2: Vibe Coding Gauntlet (2 hours)

Goal: Unlock 3 Vibe Coding badges.

1. Pick a small project (Python script or HTML page)
2. Iterate 150+ times on one file
3. Touch 300+ files (create temp files if needed)
4. Let session run 300+ messages
5. Check dashboard for: This Was Supposed To Be Quick, One More Small Change, Vibe Architect

Exercise 3: Research Rabbit Hole (1 hour)

Goal: Unlock Rabbit Hole Certified + Citation Goblin.

1. Pick a complex topic (e.g., Qdrant optimization)
2. Use `web_search` 20+ times
3. Use `web_extract` on 10+ sources
4. Use `browser_navigate` for documentation
5. Create summary with `write_file`

Exercise 4: Secret Hunter

Goal: Discover all 8 secret achievements.

See §4 Playstyle Strategy → Secret Hunter table.

7. Mastery Roadmap

Phase	Time	Target	Key Badges
-------	------	--------	------------

Beginner	Week 1–2	15+ badges, mostly Copper/Silver	First Blood, Centurion, Red Text Connoisseur, Actually Read The Logs, Memory Keeper, Rabbit Hole Certified, Toolchain Maxxer, Weekend Warrior, Night Owl
Intermediate	Week 3–6	30+ badges, Gold on favorites	All Vibe Coding, Skillsmith, Subagent Commander, Provider Polyglot, File Master, Code Executor, Terminal Tyrant
Advanced	Month 2–3	45+ badges, Diamond on core	Cron Necromancer, Full Send, Autonomous Avalanche, Tool Century, Context Dragon, Marathon Runner, Pixel Goblin, CSS Exorcist
Master	Month 4–6	55+ badges, Olympian on 10+	All Model/Provider at Olympian, all Lifestyle at Olympian, all Secrets discovered, Compression Survivor, Checkpoint Champion, Curator Caller
Legend	Month 6+	Legend badge	Olympian in ALL 59 other badges

8. Advance Track Units

For **hands-on step-by-step labs** with annotated source code and the full 23-point pedagogical structure, see the Advance Track:

Unit	Category	File
01	Usage Milestones	Unit_01.md
02	Tool Proficiency	Unit_02.md
03	Model Diversity	Unit_03.md
04	Memory & Research	Unit_04.md
05	Skills & Automation	Unit_05.md
06	Error & Recovery	Unit_06.md
07	Session Quality	Unit_07.md
08	Social & Community	Unit_08.md
09	Secret Achievements	Unit_09.md

Each unit contains:

- **Lab** (operator hands-on with exact commands)
- **Chapter** (architect analysis with source code)
- **Master Prompt** (ready-to-copy for Kimi-K2.6)
- **5 Questions** (active recall)

Appendix: Quick Reference Card

HERMES ACHIEVEMENTS CHEAT SHEET						
9 Categories		60 Badges		5 Tiers		
8 Secrets		3 Kinds		Copper→Silver→Gold→D→O		
MILESTONES	16	AUTONOMY	7	VIBE CODING	7	
MODEL LORE	5	RESEARCH	7	SKILLS	8	
LIFESTYLE	3	SOCIAL	3			

```
Dashboard: hermes dashboard → Achievements  
API: curl localhost:9119/api/plugins/hermes-achievements/  
Rescan: POST /rescan  
Share: PNG cards (1200×630) on unlocked badges
```

Last updated: June 2026 — Hermes v0.15.1. Badge data sourced from live `hermes-achievements-training.md` and GitHub repo `PCinkusz/hermes-achievements`.

Advance Track — Achievements Deep-Dive: Navigation Index

Hermes Version: v0.15.1 — Live CLI & Source Code Verified

Hardware Context: macOS Ventura, Intel x86_64 (MacBook Pro 2015, 16 GB RAM)

Status: 9 Units Complete — All 60+ Badges Covered

Total Size: ~132 KB across 9 files

Prerequisite: Units-Basic 1–5 (or equivalent CLI competence)

Date: June 2026

Quick Start

1. New to achievements?

→ Start at **Unit_01.md** — Usage Milestones.
2. Know the dashboard already?

→ Pick any unit below by category.
3. Want the secret badges?

→ Jump to **Unit_09.md** — Secret Achievements.
4. Need the full deep reference?

→ [docs/hermes-achievements-training.md](#)

Unit Map

Unit	Category	Badges	Level	Est. Time	File
01	Usage Milestones	first_blood, centurion, marathon_runner, tool_century, file_master, code_executor, terminal_tyrant, web_wanderer, vision_seeker, tts_bard, image_gen_artisan, compression_survivor, checkpoint_champion, memory_packer, curator_caller, legend	Beginner	~60 min	Unit_01.md
02	Tool Proficiency	let_him_cook, autonomous_avalanche, toolchain_maxxer, full_send, subagent_commander, background_process_enjoyer, cron_necromancer, red_text_connoisseur, stack_trace_sommelier, actually_read_the_logs, port_3000_taken, permission_denied_any_percent, dependency_hell_tourist, the_fix_was_restarting, forgot_the_env_var, yaml_colon_incident, docker_name_collision	Intermediate	~85 min	Unit_02.md
03	Model Diversity	five_model_flight, provider_polyglot, claude_confidant, gemini_cartographer, open_weights_pilgrim	Beginner	~50 min	Unit_03.md
04	Memory & Research	rabbit_hole_certified, citation_goblin, docs_archaeologist, search_orchestrator, memory_keeper, memory_palace, context_dragon	Intermediate	~75 min	Unit_04.md
05	Skills & Automation	skillsmith, skill_issue_skill_created, memory_keeper, memory_palace, context_dragon, gateway_dweller, plugin_goblin, rollback_wizard, subagent_commander, cron_necromancer, background_process_enjoyer	Intermediate	~80 min	Unit_05.md

06	Error & Recovery	<code>red_text_connoisseur</code> , <code>stack_trace_sommelier</code> , <code>actually_read_the_logs</code> , <code>port_3000_taken</code> , <code>permission_denied_any_percent</code> , <code>dependency_hell_tourist</code> , <code>the_fix_was_restarting</code> , <code>forgot_the_env_var</code> , <code>yaml_colon_incident</code> , <code>docker_name_collision</code>	Intermediate	~90 min	Unit_o6.md
07	Session Quality	<code>weekend_warrior</code> , <code>night_owl</code> , <code>early_bird_cron</code> , <code>supposed_to_be_quick</code> , <code>one_more_small_change</code> , <code>vibe_architect</code> , <code>marathon_runner</code>	Intermediate	~70 min	Unit_o7.md
08	Social & Community	<code>shared_session</code> , <code>team_delegate</code> , <code>cross_platform</code>	Beginner	~45 min	Unit_o8.md
09	Secret Achievements	<code>port_3000_taken</code> , <code>permission_denied_any_percent</code> , <code>forgot_the_env_var</code> , <code>yaml_colon_incident</code> , <code>docker_name_collision</code> , <code>rollback_wizard</code> , <code>one_character_fix</code> , <code>legend</code>	Advanced	~100 min	Unit_o9.md

Playstyle Paths

Speedrun Path (5 badges in 30 minutes)

1. **Unit_01** Lab 1 — Check dashboard, confirm `first_blood`
2. **Unit_03** Lab 3 — Switch 5 models, trigger `provider_polyglot` (Copper)
3. **Unit_04** Lab 1 — Run 5 web searches, trigger `rabbit_hole_certified`
4. **Unit_05** Lab 1 — Create one skill, trigger `skill_issue_skill_created`
5. **Unit_08** Lab 1 — Export session, trigger `shared_session`

Architect Path (Understand every engine)

1. **Unit_01** Chapter — Session counter pipeline, state.json format
2. **Unit_02** Chapter — Tool call detection, classification logic
3. **Unit_04** Chapter — Research tracker, SQL aggregation
4. **Unit_05** Chapter — Skill plugin lifecycle, cron event routing
5. **Unit_09** Chapter — Secret classifier, Legend bulk evaluation

Legend Path (Reach Olympian everywhere)

1. Complete **all 9 units** in order
2. Follow the phase roadmap in **Unit_09** §Legend Phase
3. Track with: `curl .../achievements | jq '[.achievements[] | select(.tier == "olympian")] | length'`

Structure (Every Unit)

Section	What It Contains
Lab	Operator hands-on — CLI commands, dashboard API, tier grinding
Chapter	Architect analysis — source code, engine internals, annotated pseudo-code
Master Prompt	Ready-to-copy for Kimi-K2.6 / Claude / GPT

5 Questions	Active recall — test understanding
-------------	------------------------------------

Each unit: ~250–480 lines, ~11–20 KB
Scaffolding: 10 Lab elements + 8 Chapter elements + 5 Questions = 23 points per unit
Hardware context: Referenced throughout (2015 Mac thermal limits, RAM constraints)

Related Files

File	What It Is
Units-Basic/index.md	Core Hermes training (20 Units)
hermes-achievements-training.md	Deep reference manual (500+ lines, all 60 badges with full thresholds)
Hermes-Agent-Advance-Handbook.md	Handbook overview — the complete achievement system in one file

File Tree

```
docs/training/
├── Units-Basic/           ← 20 Units (core Hermes training)
├── Units-Advance/        ← 9 Units (achievements deep-dive)
│   ├── index.md          ← THIS FILE
│   ├── Unit_01.md        ← Usage Milestones
│   ├── Unit_02.md        ← Tool Proficiency
│   ├── Unit_03.md        ← Model Diversity
│   ├── Unit_04.md        ← Memory & Research
│   ├── Unit_05.md        ← Skills & Automation
│   ├── Unit_06.md        ← Error & Recovery
│   ├── Unit_07.md        ← Session Quality
│   ├── Unit_08.md        ← Social & Community
│   └── Unit_09.md        ← Secret Achievements
├── Hermes-Agent-Advance-Handbook.md ← Overview handbook
└── hermes-achievements-training.md   ← Deep reference (60 badges)
```

Built for Thanit Tritrang. Tested on a 2015 MacBook Pro. Verified against Hermes v0.15.1. May your agent learn long and well.

Unit 01 — Usage Milestones

Scope: Lifetime counters and endgame mastery badges. **System:** Hermes Agent v0.15.1

Hardware Context: 2015 MacBook Pro, 16 GB RAM, macOS Ventura

Lab (Operator-Focused / CLI Hands-On)

1. Prerequisites

- Hermes Agent v0.15.1 running on a 2015 MacBook Pro (16 GB RAM, macOS Ventura).
- At least one completed session (`first_blood` already earned).
- Terminal with `curl` and `jq` installed.
- Dashboard reachable at `http://127.0.0.1:9119`.
- Optional: Python 3.11+ for the local tracker script.

2. Objectives

By the end of this lab you will:

- Read your Milestones & Mastery tier progress via the Dashboard API.
- Understand the exact thresholds for `centurion`, `tool_century`, `file_master`, `terminal_tyrant`, and `legend`.
- Map daily workflows to the 16 milestone badges.

3. Why It Matters

Milestone badges are **lifetime counters**. They represent the backbone of your Legend run. On a 2015 MBP with 16 GB RAM, you will hit IO and thermal limits before hitting badge limits—so knowing which workflows to batch (file ops, terminal commands, web calls) prevents throttling while grinding tiers.

4. Step-by-Step with EXACT Commands

4.1 Badge Tier Reference Table

ID	Kind	Metric	Copper	Silver	Gold	Diamond	Olympian
<code>first_blood</code>	<code>lifetime</code>	First session completed	1	—	—	—	—
<code>centurion</code>	<code>lifetime</code>	Total sessions	100	300	1,000	3,000	10,000
<code>marathon_runner</code>	<code>best_session</code>	Longest session (minutes)	60	180	480	1,200	3,000
<code>tool_century</code>	<code>lifetime</code>	Total tool calls	10k	30k	100k	300k	1M
<code>file_master</code>	<code>lifetime</code>	File operations	5k	15k	50k	150k	500k
<code>code_executor</code>	<code>lifetime</code>	<code>execute_code</code> runs	1k	3k	10k	30k	100k
<code>terminal_tyrant</code>	<code>lifetime</code>	Terminal commands	2k	6k	20k	60k	200k
<code>web_wanderer</code>	<code>lifetime</code>	Web search/extract	5k	15k	50k	150k	500k

vision_seeker	lifetime	Vision/image analysis	100	300	1,000	3,000	10,000
tts_bard	lifetime	Text-to-speech uses	50	150	500	1,500	5,000
image_gen_artisan	lifetime	Image generations	50	150	500	1,500	5,000
compression_survivor	lifetime	Context compressions	10	30	100	300	1,000
checkpoint_champion	lifetime	Checkpoints created	50	150	500	1,500	5,000
memory_packer	lifetime	Memory consolidations	10	30	100	300	1,000
curator_caller	lifetime	Curator invocations	5	15	50	150	500
legend	multi_condition	ALL metrics at Olympian	—	—	—	—	—

4.2 Check your current milestone progress

```
curl -s http://127.0.0.1:9119/api/plugins/hermes-achievements/achievements \
  | jq '.achievements[] | select(.category == "Milestones & Mastery") | {id, tier, progress, threshold}'
```

4.3 Verify First Blood

```
curl -s http://127.0.0.1:9119/api/plugins/hermes-achievements/achievements \
  | jq '.achievements[] | select(.id == "first_blood")'
```

4.4 Simulate a Centurion push Start a session and run a lightweight workload loop (safe on 2015 MBP):

```
for i in {1..25}; do
  echo "session_batch_$i" >> /tmp/hermes_centurion_log.txt
done
```

Each saved session counts toward `centurion`. On Ventura, 25 sessions can be completed in one afternoon without swap pressure.

4.5 Batch tool calls for Tool Century Inside a single Hermes session, chain small `read_file`, `search_files`, and `terminal` tasks. Each tool invocation increments the `tool_century` counter. Aim for 500+ calls across a week.

4.6 File Master grind

```
# Create a scratch directory and churn file ops
mkdir -p /tmp/hermes_file_master
for i in {1..50}; do
  touch /tmp/hermes_file_master/f_$i.txt
  echo "data" > /tmp/hermes_file_master/f_$i.txt
done
```

Use Hermes `write_file` and `patch` on these scratch files rather than project code.

4.7 Terminal Tyrant via safe loops

```
# Run 100 harmless echo commands in one Hermes terminal() call
for n in {1..100}; do echo "term_line_$n"; done
```

4.8 Web Wanderer basics

```
# Use Hermes web_search and web_extract inside a research session
# Example trigger inside Hermes:
web_search query="python asyncio best practices"
web_extract url="https://docs.python.org/3/library/asyncio.html"
```

4.9 Code Executor micro-runs

```
# Inside execute_code
for i in range(50):
    print(f"exec_batch_{i}")
```

5. Troubleshooting

Symptom	Fix
Dashboard 404	Ensure Hermes is running: <code>hermes dashboard</code>
Empty progress	Run a new session, type <code>/save</code> , then <code>POST /rescan</code>
API returns <code>null</code>	Wait for first background scan to finish (~30 s on 2015 SSD)
Legend not appearing	Secret; remains hidden until all other tiers are Olympian
High fan noise during batch	Pause 10 s between bursts; 2015 MBP thermal guard is aggressive
Swap pressure	Quit Safari tabs; use <code>terminal</code> instead of <code>browser_navigate</code>

6. Expected Outcome

You will see JSON output showing tiers from `Copper` to `Olympian` and your numeric progress. At minimum, `first_blood` shows `tier: "Copper"`.

7. Use Case Completed

A solo developer on a 2015 MBP wants to track productive weeks. Centurion Gold (1,000 sessions) proves ~2 years of daily usage.

8. Training Idea

Mount a weekly ritual: Sunday evening, run the API curl, log progress in a markdown note. Watching the numbers tick up is a powerful motivator.

9. Challenge

Reach **Silver tier** on `centurion`, `file_master`, and `terminal_tyrant` within 14 days without exceeding 80 % memory pressure on your 2015 machine.

10. Emotional Reward

The first time you see a Gold tier badge shimmer in the Dashboard, you realize the grind was the feature.

Chapter (Architect-Focused / Code & Internals)

1. Top Objectives

- Map the Milestone achievement schema to the scan engine.
- Reproduce tier-threshold evaluation logic.
- Build a local monitoring script using the public API.

2. Description

Milestones & Mastery contains 16 lifetime achievements tracked by the `hermes-achievements` plugin. Each badge uses a `lifetime` kind (cumulative) except `marathon_runner` (`best_session` / `peak minutes`) and `legend` (`multi_condition`). On low-memory hosts like a 2015 MacBook, the scan engine avoids full-history reloads by using `SNAPSHOT_TTL_SECONDS = 120` and a checkpoint cache.

3. Actions List

1. Query `/achievements` and filter by category.
2. Parse tier thresholds from the `thresholds` map.
3. Compute remaining progress: `next_threshold - current_progress`.
4. Export a weekly CSV for trend tracking.
5. Alert when a tier boundary is crossed.

4. Full Source Code Snippet with Line-by-Line Explanations

```
#!/usr/bin/env python3
# milestone_tracker.py - safe to run on a 2015 MBP with 16 GB RAM
import requests, json, csv, os

BASE = "http://127.0.0.1:9119/api/plugins/hermes-achievements"

# Line 1: Fetch the full catalog + user progress
resp = requests.get(f"{BASE}/achievements")
resp.raise_for_status()

# Line 2: Parse JSON payload
data = resp.json()

# Line 3: Filter milestone badges
milestones = [a for a in data["achievements"]
               if a.get("category") == "Milestones & Mastery"]

# Line 4: Prepare CSV rows for weekly tracking
rows = []
for badge in milestones:
    tid = badge["id"]
    progress = badge.get("progress", 0)
    tier = badge.get("tier", "None")
    # thresholds is a dict: {"copper": n, "silver": n, ...}
    thresholds = badge.get("thresholds", {})
    next_key = None
    for t in ["copper", "silver", "gold", "diamond", "olympian"]:
        if t != tier.lower() and thresholds.get(t, 0) > progress:
            next_key = t
            break
    remaining = (thresholds.get(next_key, 0) - progress) if next_key else 0
    rows.append([tid, tier, progress, next_key or "max", remaining])
```

```
# Line 5: Write CSV to disk (append mode for weekly logging)
out = os.path.expanduser("~/hermes_milestone_log.csv")
with open(out, "a", newline="") as f:
    writer = csv.writer(f)
    for r in rows:
        writer.writerow(r)

print(f"Logged {len(rows)} milestones to {out}")
```

- **Line 1:** The catalog endpoint returns achievements merged with user state.
- **Line 2:** `data["achievements"]` is a flat list of dicts.
- **Line 3:** We keep only the 16 Milestone badges to keep memory low.
- **Line 4:** Tier thresholds come from the server so the script never hard-codes values.
- **Line 5:** Appending to CSV avoids rewriting large files; ideal for spinning HDDs on older Macs.

5. Master Prompt (Ready-to-Copy for Kimi-K2.6)

You are a Hermes Agent coach. The user is working on Unit 01: Usage Milestones. They run Hermes v0.15.1 on a 2015 MacBook Pro with 16 GB RAM.

Task:

1. List the 16 Milestone badge IDs and their Copper thresholds.
2. Identify which 3 badges are easiest to reach Gold tier on a low-RAM machine.
3. Output a 14-day daily ritual (<15 min/day) that advances `centurion`, `file_master`, and `terminal_tyrant` with.
4. Include the exact curl command to check progress.
5. End with a one-line CLI alias the user can paste into `~/.zshrc`.

6. Conclusion

- Milestones reward consistency, not speed. A 2015 MBP is perfectly capable of grinding them if you batch work.
- `tool_century` and `file_master` are the two highest-count badges; optimize by using Hermes tools instead of manual CLI.
- `marathon_runner` requires session stamina, not CPU—leave a long-running research session open overnight.
- `legend` is secret and only appears after all other achievements hit Olympian; treat it as a 6–12 month horizon.
- Use the API to export CSV logs so you can visualize trends without keeping the Dashboard open.

7. Architecture Note

The plugin stores progress in `~/hermes/plugins/hermes-achievements/state.json`. On macOS Ventura with a SATA SSD, writing small JSON deltas every session keeps disk health high and avoids the heavy I/O of a full SQLite rewrite.

8. Integration Checklist

- ☐ API returns valid JSON on `curl` probe.
- ☐ `state.json` exists and updates after `/save`.
- ☐ Weekly CSV logger script runs without memory warnings.
- ☐ Dashboard renders tier icons correctly in Safari / Chrome on macOS Ventura.
- ☐ `first_blood` shows `unlocked: true`.

Questions

1. What is the `kind` tracking type for `centurion` , and how does it differ from `marathon_runner` ?
 2. How many total sessions are required to reach Olympian tier in `centurion` ?
 3. On a 2015 MacBook Pro with 16 GB RAM, why is batching tool calls preferable to many short sessions for `tool_century` ?
 4. What specific condition must be met before the `legend` badge becomes visible?
 5. What `jq` filter selects only Milestones & Mastery achievements from the `/achievements` endpoint?
-

Unit 02 — Tool Proficiency

Scope: Autonomy, debugging chaos, and toolchain intensity badges. **System:** Hermes Agent v0.15.1

Hardware Context: 2015 MacBook Pro, 16 GB RAM, macOS Ventura

Lab (Operator-Focused / CLI Hands-On)

1. Prerequisites

- Hermes Agent v0.15.1 on a 2015 MacBook Pro (16 GB RAM, macOS Ventura).
- Docker Desktop installed (older builds work on Ventura; allocate ≤ 4 GB to Docker).
- Python 3.11+ and `jq` available in `PATH`.
- At least Copper tier in `first_blood`.

2. Objectives

By the end of this lab you will:

- Trigger tool-intensity badges (`let_him_cook`, `toolchain_maxxer`, `autonomous_avalanche`).
- Safely induce controlled errors for debugging badges (`red_text_connoisseur`, `stack_trace_sommelier`, `yaml_colon_incident`).
- Run background processes and cron jobs to advance autonomy badges.

3. Why It Matters

Tool proficiency badges prove you can squeeze maximum value from Hermes on constrained hardware. A 2015 MBP's thermal envelope tightens under sustained load; learning to background tasks and batch tool calls keeps you productive without fan noise dominating the room.

4. Step-by-Step with EXACT Commands

4.1 Let Him Cook (max tool calls in one session) Inside Hermes, issue a rapid chain in a single chat:

```
read_file /tmp/test1.txt
search_files "pattern" *.py
web_search "python list comprehension"
terminal echo hello
execute_code print(2+2)
memory_get last_note
skill_manage list
```

Repeat variations until the session exceeds 200 tool calls. Use a scratch workspace to avoid project corruption.

4.2 Toolchain Maxxer (max distinct tools in one session) Explicitly invoke as many unique tool IDs as possible in one conversation:

```
1. web_search
2. web_extract
```

```

3. read_file
4. write_file
5. patch
6. search_files
7. terminal
8. execute_code
9. browser_navigate
10. memory
11. skill_manage
12. delegate_task
13. cronjob
14. todo
15. clarify
16. session_search
17. fact_store
18. image_generate

```

Aim for 18+ distinct tools in one session. On a 2015 MBP, stay under 28 distinct tools until memory pressure is comfortable.

4.3 Background Process Enjoyer

```

# Run a long tail in background via Hermes terminal(<background=True>)
terminal command="tail -f /var/log/system.log" background=True

```

Each background terminal invocation counts. On Ventura, keep ≤ 3 background logs open to preserve RAM.

4.4 Cron Necromancer

```

# Schedule a harmless cron that echoes a timestamp
cronjob schedule="*/5 * * * *" command="date >> /tmp/hermes_cron.log"

```

Wait. Each scheduled execution increments the counter. Over a week, 1k executions are easy.

4.5 Red Text Connoisseur & Stack Trace Sommelier

Deliberately trigger safe errors inside a controlled Hermes

`execute_code`:

```

# execute_code cell
1/0

```

That raises a `ZeroDivisionError`, counting toward both total errors and traceback events. Repeat 5–10 times in a sandbox notebook.

4.6 Permission Denied Any%

```

terminal command="sudo ls /root" background=False

```

On a 2015 MBP without root access, this fails with permission denied—earning the badge counter.

4.7 Port 3000 Is Taken

```

# Terminal 1: start a dummy server
python3 -m http.server 3000

```



```
# Terminal 2: try the same port again
python3 -m http.server 3000
```

The second invocation fails with port conflict. Hermes logs this if run via the `terminal` tool.

4.8 YAML Colon Incident Create a malformed YAML via Hermes `write_file`:

```
# /tmp/bad.yaml
key: value: extra_colon
```

Then run a YAML parser:

```
terminal command="python3 -c 'import yaml; yaml.safe_load(open(\"/tmp/bad.yaml\"))'"
```

The parser error counts toward the badge.

4.9 Docker Name Collision

```
terminal command="docker run --name hermes_collision_test hello-world"
terminal command="docker run --name hermes_collision_test hello-world"
```

The duplicate name triggers the Docker conflict event.

5. Troubleshooting

Symptom	Fix
Docker not found	Install Docker Desktop for Intel Mac (Ventura compatible).
<code>cronjob</code> tool missing	Ensure <code>cron</code> plugin is enabled in <code>~/hermes/config.yaml</code> .
Permission denied in Hermes terminal	The error itself is the goal; do not fix it.
Background process silently exits	Check <code>~/Library/Logs/hermes/</code> for exit codes.
Thermal throttling during tool spam	Pause 30 s between bursts; a 2015 MBP needs breath.

6. Expected Outcome

You will earn at least Copper on `let_him_cook`, `toolchain_maxxer`, `red_text_connoisseur`, and `permission_denied_any_percent`. Background and cron counters will begin incrementing.

7. Use Case Completed

A DevOps engineer on a 2015 MBP prototypes CI pipelines in Docker. Controlled failures (port collisions, permission denials) naturally populate the debugging badges while background builds advance autonomy badges.

8. Training Idea

Create a “badge farm” session template: 20 minutes of deliberate tool chaining, followed by 5 minutes of intentional errors in a sandbox, followed by scheduling three cron jobs. Run the template once per day.

9. Challenge

Reach **Gold** on `let_him_cook` (1,200 tool calls in one session) without exceeding 12 GB RAM on your 2015 MBP. Hint: use lightweight tools like `terminal` and `read_file` rather than heavy `browser_navigate` or `image_generate` calls.

10. Emotional Reward

The first time your Dashboard flashes a Diamond-tier `red_text_connoisseur`, you smile—because every crash was a lesson you chose to learn.

Chapter (Architect-Focused / Code & Internals)

1. Top Objectives

- Understand how the scan engine classifies tool calls, errors, and background events.
- Build a local dashboard widget that surfaces today’s autonomy + debugging delta.
- Design a “safe failure sandbox” that earns badges without destabilizing macOS Ventura.

2. Description

Agent Autonomy and Debugging Chaos contain 17 badges (7 + 10) focused on tool-call volume, diversity, background execution, cron usage, and a wide spectrum of error events. The plugin listens to session events in `~/.hermes/state.db` and maps each event to an achievement metric. On a 2015 MBP, the scan engine’s incremental checkpointing is critical because full SQLite rescans would block the UI for seconds.

3. Actions List

1. Open `state.db` in a read-only SQLite client to inspect raw events.
2. Map `tool_type` rows to badge categories.
3. Filter `error_message` rows to count unique error classes.
4. Export daily deltas for autonomy badges.
5. Build a macOS Notification Center script that fires on tier promotion.

4. Full Source Code Snippet with Line-by-Line Explanations

```
#!/usr/bin/env python3
# badge_delta.py — run daily on a 2015 MBP; reads API, prints deltas
import requests, json, os
from datetime import datetime

API = "http://127.0.0.1:9119/api/plugins/hermes-achievements/achievements"
SNAP = os.path.expanduser("~/hermes/badge_snapshot.json")

# Line 1: Load prior snapshot (or seed empty)
old = {}
if os.path.exists(SNAP):
    with open(SNAP) as f:
        old = json.load(f)

# Line 2: Fetch current progress
curr = {"id": a for a in requests.get(API).json()["achievements"]}
```

```
# Line 3: Compute deltas for target badges
targets = [
    "let_him_cook", "autonomous_avalanche", "toolchain_maxxer",
    "subagent_commander", "background_process_enjoyer", "cron_necromancer",
    "red_text_connoisseur", "stack_trace_sommelier",
    "port_3000_taken", "permission_denied_any_percent",
    "docker_name_collision", "yaml_colon_incident"
]

report = []
for tid in targets:
    a = curr.get(tid)
    if not a:
        continue

    prev = old.get(tid, {}).get("progress", 0)
    delta = a["progress"] - prev
    if delta > 0:
        report.append(f"{tid}: +{delta} → {a['tier']}")

# Line 4: Write new snapshot atomically
with open(SNAP, "w") as f:
    json.dump({k: {"progress": v["progress"], "tier": v["tier"]} for k,v in curr.items()}, f)

# Line 5: Print report
print(datetime.now().isoformat())
print("\n".join(report) or "No badge progress today.")
```

- **Line 1:** Storing yesterday's snapshot lets us compute daily change without querying the Dashboard UI.
- **Line 2:** The API returns a flat achievement list; we index by ID for fast lookup.
- **Line 3:** Only the 12 key badges are tracked here; expand the list as desired.
- **Line 4:** Atomic JSON write avoids corruption if the script is killed mid-save.
- **Line 5:** A concise report suitable for a macOS `say` command or Notification Center bridge.

5. Master Prompt (Ready-to-Copy for Kimi-K2.6)

You are a Hermes Agent coach. The user is working on Unit 02: Tool Proficiency. They run Hermes v0.15.1 on a 2015 MacBook Pro with 16 GB RAM.

Task:

1. List the 17 Agent Autonomy + Debugging Chaos badge IDs and their Copper thresholds.
2. Design a single 30-minute "badge farm" Hermes session that advances at least 6 distinct badges safely on
3. Indicate which tools to avoid during the session to prevent thermal throttling.
4. Provide a Python snippet that posts a macOS notification when a tier changes.
5. End with a one-line zsh alias to run the badge-delta script.

6. Conclusion

- `let_him_cook` is gated by session peak, not lifetime—use one long intense chat rather than spreading calls thin.
- `toolchain_maxxer` requires tool diversity; memorize the full tool palette to avoid duplicates.
- Background and cron badges are passive growers: set them up once, let time do the work.
- Debugging badges reward resilience on a 2015 MBP because older hardware sees more resource contention (port collisions, permission issues, Docker naming).
- The incremental scan engine keeps Dashboard performance steady even as error/event counts climb into the thousands.

7. Architecture Note

Event classification happens in `plugin_api.py`. Each row in `state.db` carries a `tool_type` and optional `error_message`. The plugin's `AchievementEvaluator` maps regex patterns (e.g., `port.*already in use`) to badge counters. Because the scan

is incremental, newly saved sessions append to the checkpoint rather than reprocessing all history.

8. Integration Checklist

- ☐ At least 18 distinct tools invoked in a single session.
 - ☐ One controlled permission-denied event logged.
 - ☐ Background process started via `terminal(background=True)`.
 - ☐ Cron job scheduled and executed successfully.
 - ☐ `badge_delta.py` prints non-empty report after a session.
-

Questions

1. What is the Copper threshold for `let_him_cook`, and what `kind` does it use?
 2. How many total distinct tools must be used in one session to reach Gold tier in `toolchain_maxxer`?
 3. On a 2015 MacBook Pro with 16 GB RAM, why should you avoid heavy tools while grinding `let_him_cook`?
 4. Which two badges in this unit are `multi_condition` rather than simple lifetime counters?
 5. What macOS file path stores the badge progress snapshot between daily delta runs?
-

Unit 03 — Model Diversity

Scope: Model & Provider Lore: multi-model sessions and provider exploration badges. **System:** Hermes Agent v0.15.1

Hardware Context: 2015 MacBook Pro, 16 GB RAM, macOS Ventura

Lab (Operator-Focused / CLI Hands-On)

1. Prerequisites

- Hermes Agent v0.15.1 on a 2015 MacBook Pro (16 GB RAM, macOS Ventura).
- Active API keys for at least 3 providers (e.g., Anthropic, Ollama Cloud, OpenRouter).
- Local Ollama installed (Intel Mac build) OR a lightweight local model endpoint.
- Familiarity with `/model` or `hermes --model` to switch providers mid-session.
- Stable internet connection capable of handling concurrent API streams on older Wi-Fi hardware.

2. Objectives

By the end of this lab you will:

- Use 5+ models in a single session to trigger `five_model_flight`.
- Register 3 distinct providers toward `provider_polyglot` Copper.
- Run model-specific sessions for `claude_confidant`, `gemini_cartographer`, and `open_weights_pilgrim`.
- Verify progress via the Achievements Dashboard API.

3. Why It Matters

Model diversity is not vanity—it improves response quality by matching the right architecture to each task. On a 2015 MacBook Pro, local models (Ollama CPU inference) are slower but incur zero token cost and work offline. Remote providers fill capability gaps without loading heavy weights into 16 GB RAM.

4. Step-by-Step with EXACT Commands

4.1 Badge Tier Reference Table

ID	Name	Kind	Metric	Copper	Silver	Gold	Diamond	Olympian
<code>five_model_flight</code>	Five-Model Flight	<code>best_session</code>	5+ models used in one session	—	—	—	—	—
<code>provider_polyglot</code>	Provider Polyglot	<code>lifetime</code>	Distinct providers used	3	5	8	12	18
<code>claude_confidant</code>	Claude Confidant	<code>lifetime</code>	Claude-specific sessions	50	150	400	1,000	3,000
<code>gemini_cartographer</code>	Gemini Cartographer	<code>lifetime</code>	Gemini-specific sessions	50	150	400	1,000	3,000
<code>open_weights_pilgrim</code>	Open Weights Pilgrim	<code>lifetime</code>	Local/OSS model sessions	50	150	400	1,000	3,000

4.2 Enumerate available providers

```
hermes providers list
```

On a 2015 MBP running Ventura, expect Ollama (local), Anthropic, and OpenRouter by default after API keys are exported:

```
export ANTHROPIC_API_KEY="sk-ant-..."
export OPENROUTER_API_KEY="sk-or-..."
```

4.2 Start a Five-Model Flight session Inside Hermes `/model` interface, switch sequentially:

```
/model claude-sonnet-4-20250514
/model kimi-k2.6
/model glm-5.1
/model qwen3-235b-a22b
/model gemini-2.5-pro
```

After each switch, send at least one message (e.g., "Summarize the previous step in one sentence"). The `best_session` metric tracks distinct models used inside a single saved session. On a 2015 machine, wait for each model to finish before switching to avoid queuing too many concurrent streams.

4.3 Provider Polyglot (lifetime distinct providers) Run one session with each provider:

```
# Session A - Anthropic
hermes --model claude-sonnet-4-20250514

# Session B - Ollama (local)
hermes --model ollama/llama3.1:8b

# Session C - OpenRouter / DeepSeek
hermes --model openrouter/deepseek-chat
```

Each distinct provider increments the lifetime counter. Copper requires only 3; Gold requires 8, which means exploring every provider Hermes supports.

4.4 Claude Confidant Set Claude as the default model for a week:

```
hermes --model claude-sonnet-4-20250514
```

Each saved session counts. Copper is 50 sessions; on a daily driver workflow, that takes roughly two months. Speed-run by running short `read_file` or `terminal` micro-sessions and typing `/save`.

4.5 Gemini Cartographer

```
hermes --model gemini-2.5-pro
```

Same pattern as Claude Confidant. Google's Gemini is especially useful for long-context tasks (up to 1M tokens), which complements the 2015 MBP's limited RAM.

4.6 Open Weights Pilgrim

```
# Start local Ollama (CPU inference on 2015 dual-core i7)
ollama run llama3.1:8b

# Then open a Hermes session targeting local Ollama endpoint
hermes --model ollama/llama3.1:8b
```

each saved session with a local/OSS model increments the counter. Note: on a 2015 MBP, stick to 7B–8B parameter models. Running 70B via CPU would swap aggressively and overheat.

4.7 Verify progress via API

```
curl -s http://127.0.0.1:9119/api/plugins/hermes-achievements/achievements \
  | jq '.achievements[] | select(.category == "Model & Provider Lore") | {id, tier, progress, thresholds
```

5. Troubleshooting

Symptom	Fix
<code>/model</code> returns "provider not found"	Check API keys in environment and <code>~/hermes/config.yaml</code> .
Local Ollama runs out of RAM	Use 7B or 8B models only; quit Safari tabs to free memory.
Five-Model Flight not triggering	Ensure all 5 switches occur inside the same saved session.
Long latency on 2015 Wi-Fi	Switch off Bluetooth or use Ethernet dongle to reduce interference.
Dashboard shows <code>null</code> tier	Wait for the next background scan after <code>/save</code> .

6. Expected Outcome

You will see at least one model-specific badge at Copper tier and `provider_polyglot` at Copper (3 providers). Five-Model Flight will appear as Unlocked if the session included 5+ distinct models.

7. Use Case Completed

A researcher on a 2015 MBP uses Claude for analysis, Gemini for long-document summarization, and local Llama for offline brainstorming. Tracking Model Diversity badges gamifies responsible multi-model habits.

8. Training Idea

Create a weekly "Model Rotation Tuesday": alternate default models every Tuesday. This naturally pushes `claude_confidant`, `gemini_cartographer`, and `open_weights_pilgrim` forward in parallel.

9. Challenge

Reach **Gold** on `provider_polyglot` (8 providers) and **Copper** on all three model-specific badges within 60 days on a 2015 MacBook Pro with 16 GB RAM and no GPU.

10. Emotional Reward

The first time you switch from a cloud model to a local 8B model on a plane with no Wi-Fi—and it still answers beautifully—you understand why `open_weights_pilgrim` exists.

Chapter (Architect-Focused / Code & Internals)

1. Top Objectives

- Map the Model & Provider Lore schema to provider IDs and model family strings.
- Build a local session logger that records which model handled each turn.
- Automate provider-switch prompts to guarantee Five-Model Flight in long sessions.

2. Description

Model & Provider Lore contains 5 achievements: one `best_session ( five_model_flight )` and four `lifetime` counters (`provider_polyglot`, `claude_confidant`, `gemini_cartographer`, `open_weights_pilgrim`). The plugin parses `state.db` rows for `model_id` and `provider_id`. On Hermes v0.15.1, the evaluator normalizes model strings (e.g., `kimi-k2.6`, `glm-5.1`, `qwen3-235b-a22b`, `gemini-2.5-pro`) into a per-session set. If that set's cardinality reaches 5, the badge fires. For lifetime counters, each session increments the tally for its active provider ID and, when applicable, the model-family bucket.

3. Actions List

1. Query `/achievements` and filter for Model & Provider Lore.
2. Read `~/hermes/state.db` (SQLite) to inspect `model_id` and `provider_id` per message.
3. Build a CLI helper that prepends `/model <name>` to a batch of prompts.
4. Export a monthly chart of provider usage to guide cost optimization.
5. Write a cronjob that reminds you to switch models if 3 consecutive sessions used the same provider.

4. Full Source Code Snippet with Line-by-Line Explanations

```
#!/usr/bin/env python3
# model_flight_helper.py - automate provider switching inside a Hermes session
import json, os

# Step 1: Define the 5-model roster optimized for a 2015 MBP
# (4 cloud models + 1 lightweight local model)
ROSTER = [
    "claude-sonnet-4-20250514",
    "kimi-k2.6",
    "gemini-2.5-pro",
    "openrouter/deepseek-chat",
    "ollama/llama3.1:8b"
]

# Step 2: Build a "prompt deck" - one message per model
workload = "Explain the concept of incremental scan caching in 2 sentences."
prompts = []
for idx, model in enumerate(ROSTER, start=1):
    # Each prompt prefixed with /model so Hermes switches providers
    prompts.append(f"/model {model}\n{workload} (turn {idx}/5)")

# Step 3: Write prompts to a batch file for copy-paste into Hermes TUI
batch_path = os.path.expanduser("~/hermes_five_model_batch.txt")
with open(batch_path, "w") as f:
    f.write("\n---\n".join(prompts))

print(f"Batch written to: {batch_path}")
print(f"Models in flight: {len(ROSTER)}")
print("Instructions: Paste each block (between --- delimiters) into Hermes.")
```


- **Step 1:** The roster blends remote APIs (fast, high capability) with one local model (offline, low RAM). On a 2015 MBP, the local model is intentionally 8B to avoid CPU thrashing.
- **Step 2:** Prefixing `/model <name>` forces Hermes to switch before processing the workload. Each switch counts toward the session's distinct-model set.
- **Step 3:** Writing to a flat file lets the user paste sequentially without memorizing commands. The `---` delimiter makes chunking obvious in any TUI.

5. Master Prompt (Ready-to-Copy for Kimi-K2.6)

You are a Hermes Agent coach. The user is working on Unit 03: Model Diversity.
They run Hermes v0.15.1 on a 2015 MacBook Pro with 16 GB RAM.

Task:

1. List the 5 Model & Provider Lore badge IDs, their kinds, and their thresholds (all tiers).
2. Recommend a 5-model roster that works well on a 2015 MBP (intel CPU, 16 GB RAM, no GPU).
3. Provide a single-session transcript template that triggers Five-Model Flight without exceeding RAM.
4. Write a Python snippet that queries the API and prints the top under-leveled model-specific badge.
5. End with a one-line zsh alias to launch Hermes with the least-used provider today.

6. Conclusion

- `five_model_flight` rewards breadth in a single session; plan your roster ahead of time rather than improvising switches.
- `provider_polyglot` is lifetime-cumulative; every new provider you try permanently advances the counter.
- On a 2015 MBP, local models are viable but capped at ~8B parameters; use cloud models for heavy reasoning.
- `claude_confidant`, `gemini_cartographer`, and `open_weights_pilgrim` encourage loyalty to specific model families. Rotate weekly to avoid over-investing in one badge while neglecting others.
- The plugin's model-family normalization means `gemini-2.5-pro` and future Gemini versions may count toward the same cartographer bucket—check release notes when new models ship.

7. Architecture Note

Provider metadata is stored in `state.db` as JSON columns on the `messages` table. The achievements plugin scans these rows incrementally and hashes `(session_id, provider_id, model_id)` tuples to avoid double-counting restarts or edits. Because disk I/O on a 2015 SATA SSD is modest, the plugin caps concurrent scans to one background thread.

8. Integration Checklist

- ☐ At least 3 distinct providers show `unlocked: true` in Dashboard API.
- ☐ One session contains messages from 5+ distinct models in `state.db`.
- ☐ `provider_polyglot` lifetime counter increments after trying a new provider.
- ☐ Local Ollama endpoint responds within 30 seconds on `ollama/llama3.1:8b`.
- ☐ API tier query returns non-null for all Model & Provider badges.

Questions

1. What is the exact `kind` tracking type for `five_model_flight`, and what condition triggers it?
2. How many distinct providers are required to reach Diamond tier in `provider_polyglot`?
3. On a 2015 MacBook Pro with 16 GB RAM, why is the local model in the 5-model roster capped at 8B parameters?
4. Which three badges in this unit are family-specific (Claude, Gemini, Open Weights)?

5. What JSON column in `state.db` stores the model and provider metadata scanned by the plugin?

Unit 4: Memory & Research — Web Search, Docs Archaeology, and Persistent Memory

Badges covered: `rabbit_hole_certified`, `citation_goblin`, `docs_archaeologist`, `search_orchestrator`, `memory_keeper`, `memory_palace`, `context_dragon`

Lab 4: Research Deep-Dive + Memory Persistence Marathon

Level: Advanced

Estimated Time: ~90 minutes

Prerequisites: Units 1–3 complete (CLI/TUI, model switching, basic memory).

Lab Objectives

By the end of this lab you will be able to:

- Perform 20+ sequential `web_search` calls and 10+ `web_extract` calls to unlock `rabbit_hole_certified` and `citation_goblin`.
- Navigate documentation with `browser_navigate` and `read_file` to score `docs_archaeologist`.
- Trigger 50+ web calls in a single session for `search_orchestrator`.
- Write structured memories with the `memory` tool to progress `memory_keeper` and `memory_palace`.
- Trigger context-compression and memory consolidation cycles for `context_dragon`.
- Inspect your `~/.hermes/state.db` and achievement progress via the dashboard API.

Why This Lab Matters

On a 2015 MacBook Pro with 16 GB RAM, every web lookup is orchestrated in the cloud — your machine is the conductor, not the orchestra. This lab teaches you how to turn Hermes into a research machine that remembers everything it learns. The badges here represent the core competency of any autonomous agent: information retrieval + persistent recall. Without research skills, the agent is blind. Without memory, it is amnesiac.

Step-by-Step Instructions

Step 1: Open a Fresh Session and Set a Research Goal (5 minutes)

```
hermes --new
```

Type:

```
We are doing an advanced research marathon. Our topic: "Vector database comparison in 2026 — Qdrant vs Milv
```

Step 2: Execute 20 Web Searches (25 minutes)

Run these exact commands inside the chat, one by one:

```
web_search: "Qdrant vector database 2026 benchmarks"
```

```
web_search: "Milvus 2026 performance comparison"
```

```
web_search: "Weaviate vs Qdrant latency benchmark"
```

```
web_search: "pgvector PostgreSQL vector extension limitations 2026"
```

```
web_search: "ANN algorithm HNSW vs IVF 2026"
```

Continue until you hit at least 20 distinct `web_search` calls. Vary queries by adding "architecture", "pricing", "scalability", "on-premise", "managed cloud", and "RAM usage".

Troubleshooting: If a search returns no results, rephrase keywords. If your Ollama Cloud rate limit triggers, pause 5 minutes and send a non-web message.

Step 3: Extract 10 Sources with `web_extract` (20 minutes)

For the top 10 URLs returned in Step 2, run:

```
web_extract: "https://qdrant.tech/documentation/..."
```

Type the exact URL. Repeat 10 times. Aim for doc pages, blog posts, and GitHub READMEs.

Step 4: Read Local Documentation Files (10 minutes)

From Terminal (outside chat), find any docs already on disk:

```
find ~ -type f -name "*.md" | head -30
```

Back in chat, run `read_file` on 5+ long Markdown files:

```
read_file: "/Users/thanit/work/hermes-dev/hermes-learn/docs/hermes-achievements-training.md"
```

Each `read_file` counts toward `docs_archaeologist`. On a 2015 MacBook, read local files first — they are instant and do not consume cloud quota.

Step 5: Write Memory Entries After Each Insight (10 minutes)

After every 3-4 searches, type:

```
memory: {"action": "save", "category": "research", "content": "Qdrant uses HNSW by default with configurabl
```

Vary categories: `research`, `hardware`, `vector_db`, `pricing`. Write at least 8 memory entries.

Step 6: Compress Context and Verify Dashboard (10 minutes)

Trigger context compression manually:

```
Compress my session context now.
```

Exit the chat (`/quit`) and check progress:

```
curl -s http://127.0.0.1:9119/api/plugins/hermes-achievements/achievements | jq '.achievements[] | select'
```

Step 7: Resume and Test Recall (5 minutes)

```
hermes --continue
```

Ask: What vector databases did we research and what did we conclude about Qdrant on 16GB RAM?

If memory works, Hermes will parrot back stored entries.

Expected Outcome

- 20+ web calls completed (progress toward Rabbit Hole Certified — Copper is 400 lifetime, but 20 is a strong start).
- 10+ `web_extract` calls completed.
- 5+ documentation files read.
- 8+ memory entries saved.
- Dashboard shows upward progress on at least 5 badges.
- Agent correctly recalls research conclusions after resuming.

Lab 4 Use Case Completed

You treated Hermes as a dedicated research analyst. It searched, extracted, read docs, and wrote memories — all while you monitored progress on a real badge board. Your 2015 MacBook Pro handled the orchestration load without breaking a sweat.

Training Idea / Self-Improvement Focus

- The Curator will scan this session in 7 days. If the memory entries are structured (category + timestamp + content), it may extract a reusable `vector_db_research` skill.
- `context_dragon` unlocks with 5,000 lifetime context events. Frequent compression and long sessions accelerate this.
- Consider using `delegate_task` for parallel research threads in Unit 5.

Challenge (Recommended)

1. In one session, hit 50 web calls for `search_orchestrator` Copper (50 max in one session).
2. Create a memory entry after EVERY single web call.
3. Export your memory store:

```
sqlite3 ~/.hermes/memory_store.db ".dump" > ~/research_memory_dump.sql
```

4. Import it into a spreadsheet and count how many categories you used.

Emotional Reward

🏆 You just turned a 2015 MacBook into a research engine with a photographic memory. Every search, every extract, every doc read is now a badge on your wall.

Chapter 4: The Research & Memory Stack — How Hermes Never Forgets

Top Ideal Study Objectives:

- Trace a web call from the CLI through the search module to the model response.
- Understand how `memory` tool writes to both SQLite and HRR vectors.
- See why local docs (`read_file`) and remote docs (`browser_navigate`) share the same pipeline.
- Connect context compression to `context_dragon` progress.

Description: This chapter is the architect-level view of the research and memory subsystems. Lab 4 was operator-focused: you ran the tools. This chapter explains the wiring: how `web_search` resolves to a provider, how `memory` persists, and why your old MacBook can offload network I/O without blocking the event loop.

Actions List (Topics):

1. Locate the web search adapter in the source tree.
2. Trace a `memory` save from CLI to SQLite + HRR storage.
3. Read the context compression pseudocode.
4. Map each badge metric to the exact function that increments it.

Full Source Code Snippets with Line-by-Line Explanations:

```
# research_memory.py -- Educational pseudocode for the web + memory pipeline
# Version: v0.15.1+

"""
Research & Memory orchestrator.

This module handles:
- web_search, web_extract, browser_navigate
- memory save / recall / list
- docs_archaeologist counting (local doc reads)
- context compression triggering

It is fully async so a 2015 MacBook Pro does not block Terminal
while waiting for Ollama Cloud model responses or HTTP downloads.
"""

import asyncio          # Line ~10 -- non-blocking I/O for all web calls
import sqlite3          # Line ~11 -- local state persistence
import json             # Line ~12 -- memory payload serialization
from typing import Dict # Line ~13 -- type safety for memory records

# Line-by-line:
# asyncio : Every web_search is wrapped in an asyncio Task. This means
#           you can fire 10 searches and they multiplex over a single
#           event loop — critical on dual-core 2015 CPUs.
# sqlite3 : Memory and achievements are stored locally. No cloud DB
#           required. Your data stays on your SSD.
# json     : Memory entries are JSON blobs with schema:
#           {category, content, created_at, source_session_id}
# typing   : Enforces contract between tools and storage layers.

# ---- Memory save implementation ----
# Called by the memory tool after every approved save.

async def save_memory(payload: Dict[str, str]) -> str:
    conn = sqlite3.connect("~/hermes/memory_store.db") # Line ~25
    cursor = conn.cursor()                             # Line ~26
    cursor.execute("""
        INSERT INTO memories (category, content, created_at)
```

```

        VALUES (?, ?, datetime('now'))
        """ , (payload["category"], payload["content"]))          # Line ~30
    conn.commit()                                                  # Line ~31
    conn.close()                                                  # Line ~32
    return f"Memory saved: {payload['category']}"                 # Line ~33

# Line ~25 : Opens the SQLite DB. On first run it auto-creates tables.
# Line ~30 : Parameterized query prevents SQL injection from any model
#             output that might attempt prompt-injection.
# Line ~31 : commit() is synchronous but the connection is local SSD.
#             On a 2015 MacBook Pro SSD, this is <1ms.
# Line ~33 : Returns confirmation to the model so it knows the save
#             succeeded and can decide to reference it later.

# ---- Context compression trigger ----
# Called when session message count exceeds threshold (default ~120).

async def maybe_compress(session_messages: list) -> list:
    if len(session_messages) > COMPRESS_THRESHOLD:              # Line ~40
        summary = await model.summarize(session_messages)        # Line ~41
        return [{"role": "system", "content": summary}]          # Line ~42
    return session_messages                                      # Line ~43

# Line ~40 : Threshold is tunable in config.yaml under
#             memory.compression_threshold.
# Line ~41 : Offloads summarization to Ollama Cloud so the local CPU
#             is not burdened with long-context inference.
# Line ~42 : Replaces the long message list with a single summary.
#             This is what unlocks context_dragon progress.

```

Master Prompt for This Chapter:

Copy and paste into Kimi-K2.6 / Claude / GPT:

You are continuing my Hermes Agent deep-dive education from Chapter 4.
 Explain the research and memory architecture:

1. Why is asyncio crucial for web_search on a 2015 MacBook Pro?
2. How does save_memory prevent SQL injection from model outputs?
3. What is the exact JSON schema of a memory payload?
4. When does maybe_compress trigger, and how does it relate to debugging large sessions on old hardware?
5. Which badge metrics are incremented after web_search vs read_file?

Conclude with exactly 5 questions to test my understanding.

Chapter Conclusion — What You Have Learned

1. `web_search` and `memory` share the same async event loop, which is why your Terminal stays responsive on a 2015 MacBook.
2. SQLite is the ground truth for memory, but future retrieval uses HRR (Holographic Reduced Representation) vectors for semantic similarity.
3. Context compression is not just a memory optimization — it is a tracked metric (`context_dragon`) that gamifies good session hygiene.
4. Local file reads (`read_file`) and browser navigation (`browser_navigate`) both count toward documentation badges because they represent "information foraging" behavior.
5. The achievement scanner reads `~/hermes/state.db` incrementally, so every badge you earn updates sub-second.

5 Questions to Check Your Understanding:

1. If you ran 10 `web_search` calls synchronously on a 2015 dual-core MacBook, what would happen to Terminal responsiveness? How does asyncio solve this?

2. Why does `save_memory` use parameterized SQL `(?, ?, ?)` instead of Python f-strings? What threat model does this mitigate?
 3. `docs_archaeologist` tracks documentation reads. Does `read_file` on a local README count the same as `browser_navigate` to a remote docs page? Explain the scanner logic.
 4. Your session hit 130 messages and `maybe_compress` triggered. What is the output of the function and how does it affect the achievement scanner's view of `context_dragon`?
 5. You want to export all memories in category "research" to a JSON file. Write the exact sqlite3 command a 2015 MacBook user would run in Terminal.
-

Unit 5: Skills & Automation — Procedural Memory, Plugins, Cron, and Subagents

Badges covered: `skillsmith`, `skill_issue_skill_created`, `memory_keeper`, `memory_palace`, `context_dragon`, `gateway_dweller`, `plugin_goblin`, `rollback_wizard`, `subagent_commander`, `cron_necromancer`, `background_process_enjoyer`

Lab 5: Automation Gauntlet — Skills, Plugins, Cron, and Delegated Subagents

Level: Advanced

Estimated Time: ~100 minutes

Prerequisites: Units 1–4 complete (memory, research, model switching).

Lab Objectives

By the end of this lab you will be able to:

- Create a new skill from scratch using the `skill_manage` tool to unlock `skillsmith` and `skill_issue_skill_created`.
- View and manage skills via `hermes skills list` and `/skills`.
- Install a plugin from the community registry to progress `plugin_goblin`.
- Trigger a rollback with the `/rollback` command to unlock `rollback_wizard`.
- Spawn a delegated subagent with `delegate_task` to progress `subagent_commander`.
- Create a cron job using the `cronjob` tool to advance `cron_necromancer`.
- Run a background process via `terminal(background=True)` for `background_process_enjoyer`.
- Send a message through the gateway (Telegram/Discord/Slack) if configured for `gateway_dweller`.

Why This Lab Matters

This is the "let the agent run independently" lab. On a 2015 MacBook Pro, background processes and cron jobs are how you keep Hermes productive while you sleep. Skills are procedural memory — the agent remembers HOW to do things. Plugins extend what Hermes can touch. Subagents parallelize work. This lab turns a chatbot into an autonomous system.

Step-by-Step Instructions

Step 1: Skill Forge — Create a Reusable Skill (15 minutes)

Start a fresh session:

```
hermes --new
```

Type this exact prompt:

```
Create a skill named "macbook_health_check" that:  
1. Runs a Python script to check CPU temperature via osx-cpu-temp
```

```
2. Checks free RAM with psutil
3. Checks free disk space with shutil.disk_usage
4. Returns a formatted health report
Save it with skill_manage.
```

Approve all tool calls. When it finishes, verify:

```
hermes skills list | grep macbook
```

Check `/skills` inside the chat as well.

Troubleshooting: If `skill_manage` is not available, the agent may have auto-extracted the skill. Look for it in `~/.hermes/skills/macbook_health_check.md`. If missing, create it manually:

```
cat > ~/.hermes/skills/macbook_health_check.md << 'EOF'
---
name: macbook_health_check
description: Checks 2015 MacBook Pro health metrics
trigger: "health", "thermal", "ram", "disk"
version: 1.0
---

1. Run Python to collect metrics
2. Format a human-readable report
3. Suggest actions if metrics are concerning
EOF
```

Step 2: Trigger Skill Reuse (5 minutes)

In the same session, type:

```
Run a health check on my MacBook.
```

If the skill triggers correctly, the agent should ask to execute the Python script without you describing it again. This is `skillsmith` in action.

Step 3: Install a Plugin (10 minutes)

From Terminal:

```
hermes plugin list
```

Pick any available plugin (e.g., `web_search_advanced`, `screenshot`, `code_runner`). Install it:

```
hermes plugin install screenshot
```

Verify with:

```
hermes plugin list | grep screenshot
```

Each install counts toward `plugin_goblin` (Copper at 1,000 lifetime plugin events). If you are already past Copper, every new install edges you closer.

Step 4: Rollback After a Bad Edit (10 minutes)

Inside the chat, intentionally request a destructive change:

```
Delete every file in ~/tmp_test_folder/. Do it now.
```

(Only do this if `~/tmp_test_folder/` exists and contains disposable files.)

After confirming deletion, type:

```
/rollback
```

Hermes should restore the last checkpoint. Repeat rollbacks 5+ times across the session.

Alternative path if rollback is unavailable: Use the `patch` tool to make a change, then revert it with `patch` again. Some scanners count manual patches toward `rollback_wizard`.

Step 5: Delegate a Task to a Subagent (15 minutes)

Type:

```
delegate_task: {
  "task": "Research the current top 3 Python web frameworks in 2026. Return a bullet summary.",
  "timeout": 120,
  "tools": ["web_search", "web_extract"]
}
```

Approve the delegation. Wait for the subagent to return.

Repeat with a second task:

```
delegate_task: {
  "task": "Summarize our memory entries in category 'research'.",
  "timeout": 60,
  "tools": ["memory"]
}
```

Every `delegate_task` counts toward `subagent_commander` (Copper at 5 lifetime).

Step 6: Create a Cron Job (15 minutes)

From Terminal:

```
hermes cron list
```

Add a simple cron job:

```
hermes cron add --name "daily_log" --schedule "0 9 * * *" --command "echo 'Hermes alive at $(date)' >> ~
```

Verify it appears in the list. This advances `cron_necromancer` (Copper at 1,000 lifetime cron calls).

On a 2015 MacBook, cron jobs are lightweight. Even an overworked old CPU can handle a single echo every morning.

Step 7: Run a Background Process (10 minutes)

Inside the chat, type:

```
terminal: {"command": "ping -c 20 8.8.8.8", "background": true}
```

Hermes should return a session ID. Poll it a minute later:

```
Check background process status for the session ID you gave me.
```

This counts toward `background_process_enjoyer` (Copper at 300 lifetime).

Step 8: Memory Housekeeping (5 minutes)

Write 3 structured memory entries:

```
memory: {"action": "save", "category": "automation", "content": "Installed screenshot plugin and tested cro
```

```
memory: {"action": "save", "category": "automation", "content": "Delegated two subagent tasks: Python frame
```

```
memory: {"action": "save", "category": "automation", "content": "Created macbook_health_check skill v1.0 an
```

These feed `memory_keeper` + `memory_palace`.

Expected Outcome

- At least 1 new skill created and verified in `~/hermes/skills/`.
- At least 1 plugin installed and visible in `hermes plugin list`.
- At least 5 `delegate_task` calls completed (target: Copper).
- At least 1 cron job scheduled.
- At least 1 background process executed.
- 3+ memory entries saved.
- Dashboard shows increased progress on multiple automation badges.

Lab 5 Use Case Completed

You transformed Hermes from a chat interface into an automation platform. It has procedural memory (skills), extensibility (plugins), scheduling (cron), parallelism (subagents), and background execution (terminal). On a 2015 MacBook Pro, this means the machine works while you sleep.

Training Idea / Self-Improvement Focus

- Skills you create in this lab may be promoted by the Curator if they are reused frequently.
- Cron jobs run in the macOS cron daemon. Check `/usr/bin/crontab -l` to see Hermes entries.
- Background processes use asyncio Tasks under the hood. On Ventura, you can inspect them with `ps -ax | grep hermes`.
- Subagent delegation isolates context — each subagent gets its own memory slice. This protects your main session from context pollution.

Challenge (Recommended)

1. Chain a cron job to a skill: schedule `macbook_health_check` to run every morning at 08:00 and write the result to a memory entry.
2. Delegate 5 tasks in a single session to hit `subagent_commander` Copper in one shot.
3. Check if the gateway is active: type `/gateway status` and send a test message to Telegram/Discord if configured.
4. Export your skill to Git:

```
cd ~/.hermes/skills && git init && git add . && git commit -m "Lab 5 skills"
```

Emotional Reward

🏆 Your 2015 MacBook Pro now runs an autonomous agent empire. Skills, plugins, cron jobs, subagents, and background processes — you are no longer chatting. You are commanding.

Chapter 5: The Automation Stack — Skills, Curator, and Plugin Lifecycle

Top Ideal Study Objectives:

- Trace a skill from markdown file to execution trigger.
- Understand the Curator's 7-day review cycle for skill promotion/pruning.
- Map plugin events to the `plugin_goblin` metric.
- See how cron, background processes, and subagents share the same asyncio loop.

Description: This chapter reveals the architecture beneath Lab 5. We explore how skills become reusable, how the Curator decides what stays and what goes, and why every plugin install is tracked. If Lab 5 was operating the controls, this chapter is reading the schematics.

Actions List (Topics):

1. Read a skill Markdown file from `~/.hermes/skills/`.
2. Find the Curator's schedule in `~/.hermes/config.yaml`.
3. Inspect `plugin_api.py` for install hooks.
4. Trace `delegate_task` isolation in the source tree.

Full Source Code Snippets with Line-by-Line Explanations:

```
# automation_stack.py -- Educational pseudocode for skills, plugins, cron, subagents
# Version: v0.15.1+

"""
Automation Stack orchestrator.

Handles:
- Skill load / parse / trigger matching
- Plugin install / uninstall / event hooks
- Cron job registration via macOS crontab
- Subagent fork with isolated context
- Background process delegation

Everything runs on the same asyncio event loop so a 2015 MacBook
with 16GB RAM does not spawn a new OS process per subagent.
"""
```

```

import asyncio                # Line ~15 -- shared loop for all async work
import yaml                   # Line ~16 -- frontmatter parsing in skills
import sqlite3                # Line ~17 -- plugin state persistence
import subprocess             # Line ~18 -- cron and background shell
from pathlib import Path       # Line ~19 -- skill path resolution

# Line-by-line:
# asyncio : Subagents and background processes are asyncio Tasks, not
#           OS processes. This keeps RAM usage below ~500MB even with
#           5 concurrent subagents on a 2015 MacBook.
# yaml : Skill files use YAML frontmatter (--- \n key: value ...). The
#        parser extracts triggers, version, and description before
#        reading the markdown body.
# sqlite3 : Plugin metadata (installed, enabled, version) is stored in
#           plugin_registry.db so Hermes can list and track events.
# subprocess : Cron uses the macOS crontab command. Background shell jobs
#              use subprocess.Popen so output can be streamed back.
# Path : Skills are resolved relative to ~/.hermes/skills/. Path
#        ensures cross-platform compatibility (macOS Ventura here).

# ---- Skill trigger matching ----
# Called by the agent loop when user speech matches a skill trigger.

def match_skill(user_input: str, skills_dir: Path) -> dict | None:
    for skill_path in skills_dir.glob("*.md"): # Line ~30
        with open(skill_path, "r") as f:
            frontmatter = yaml.safe_load(f.read()) # Line ~32
            triggers = frontmatter.get("trigger", "") # Line ~33
            if any(t in user_input.lower() for t in triggers.split(",")):
                return frontmatter # Line ~35
    return None # Line ~36

# Line ~30 : Iterates over every .md file in ~/.hermes/skills/. On a
#           2015 MacBook, 50 skills load in <10ms from SSD.
# Line ~33 : Trigger is a comma-separated string in YAML. Example:
#           trigger: "health, thermal, ram, disk"
# Line ~35 : Returns the entire skill definition so the agent can
#           inject the body into the next model prompt.

# ---- Plugin install hook ----
# Called by `hermes plugin install <name>`.

async def install_plugin(name: str) -> str:
    url = f"https://agentskills.io/plugins/{name}.zip" # Line ~42
    download = await asyncio.to_thread(_download, url) # Line ~43
    extract_path = Path(f"~/.hermes/plugins/{name}") # Line ~44
    await asyncio.to_thread(_extract, download, extract_path) # Line ~45
    _record_plugin_event(name, "install") # Line ~46
    return f"Plugin {name} installed." # Line ~47

# Line ~42 : Community plugins are hosted on agentskills.io. The URL
#           pattern is stable.
# Line ~43 : asyncio.to_thread() moves blocking network I/O off the
#           main event loop. This prevents UI freeze on old Macs.
# Line ~46 : _record_plugin_event increments the plugin_goblin metric
#           in state.json. Every install, enable, disable counts.

# ---- Cron registration (macOS) ----
# Writes a crontab line for the Hermes user.

def add_cron(name: str, schedule: str, cmd: str) -> None:
    entry = f"{schedule} {cmd} # hermes-cron:{name}\n" # Line ~54
    existing = subprocess.getoutput("crontab -l") # Line ~55
    if name not in existing: # Line ~56
        new_crontab = existing + "\n" + entry # Line ~57
        subprocess.run(["crontab", "-"], input=new_crontab, text=True) # Line ~58

```

```

# Line ~54 : Hermes tags each crontab line with a comment so it can
#             identify its own entries later. Example:
#             0 9 * * * echo ... # hermes-cron:daily_log
# Line ~58 : Uses stdin to pipe the new crontab. This is safer than
#             appending to a file directly.

# ---- Subagent isolation ----
# delegate_task creates a new HermesState with a blank memory slice.

async def fork_subagent(task: str, tools: list, timeout: int) -> str:
    child_state = HermesState.new_clone(parent=MainState) # Line ~65
    child_state.tool_filter = tools                       # Line ~66
    child_state.memory_slice = {}                         # Line ~67
    result = await asyncio.wait_for(
        run_agent_loop(child_state, task), timeout=timeout # Line ~69
    )
    return result["summary"]                             # Line ~71

# Line ~65 : new_clone copies config but NOT session history. This
#             ensures the subagent does not leak private context.
# Line ~66 : tool_filter restricts which tools the subagent may call.
#             If you only pass ["web_search"], the subagent cannot
#             write files or run terminal commands.
# Line ~67 : Blank memory slice prevents the subagent from reading
#             your entire memory store.
# Line ~69 : asyncio.wait_for enforces the timeout. If the subagent
#             hangs, it raises asyncio.TimeoutError and returns a
#             partial summary.

```

Master Prompt for This Chapter:

You are continuing my Hermes Agent deep-dive education from Chapter 5. Explain the automation stack architecture:

1. How does skill trigger matching work and why is YAML frontmatter parsed before markdown body?
2. What prevents a subagent from accessing the parent session's private memories? Trace the isolation mechanism.
3. Why does `install_plugin` use `asyncio.to_thread` for download?
4. How does Hermes identify its own crontab entries on macOS?
5. Which badges are incremented by skill creation, plugin install, cron add, `delegate_task`, and background process execution?

Conclude with exactly 5 questions to test my understanding.

Chapter Conclusion — What You Have Learned

1. Skills are just Markdown with YAML. The agent parses triggers at load time and matches against lowercase user input. There is no magic — just fast string matching.
2. Subagent isolation is a hard boundary: `new_clone()` copies config, not context. This is why you can safely delegate untrusted tasks.
3. `asyncio.to_thread()` is the secret sauce that keeps your 2015 MacBook responsive during network downloads and heavy operations.
4. Cron jobs are plain macOS crontab entries with Hermes tags. You can read them with `crontab -l` and edit them manually if needed.
5. The automation badge suite (skillsmith, plugin_goblin, cron_necromancer, subagent_commander, background_process_enjoyer) is tracked by the same scanner engine that reads state.db incrementally.

5 Questions to Check Your Understanding:

1. You have 200 skills in `~/hermes/skills/`. Why does skill matching still take <50ms on a 2015 MacBook, and what hardware factor (SSD vs HDD) makes this possible?

2. A subagent is given `tools: ["web_search", "web_extract"]`. It tries to call `write_file`. What happens and why?
 3. You run `hermes plugin install heavy_computation`. The download is 500 MB. Why does your Terminal not freeze, and which Python mechanism ensures this?
 4. You see this crontab line: `0 9 * * * /usr/local/bin/hermes run --skill daily_log # hermes-cron:daily_log`. If you delete the comment, what breaks in `hermes cron list`?
 5. If `subagent_commander` requires 5 lifetime `delegate_task` calls, and each call forks a new `HermesState`, does the parent session's `memory_store.db` grow with each call? Explain.
-

Unit 6: Error & Recovery — Debugging Chaos, Secret Badges, and Resilience

Badges covered: `red_text_connoisseur`, `stack_trace_sommelier`, `actually_read_the_logs`, `port_3000_taken`, `permission_denied_any_percent`, `dependency_hell_tourist`, `the_fix_was_restarting`, `forgot_the_env_var`, `yaml_colon_incident`, `docker_name_collision`

Lab 6: Debugging Chaos — Embracing Errors as Achievements

Level: Advanced

Estimated Time: ~90 minutes

Prerequisites: Units 1–5 complete (comfortable with terminal, web, file, and Docker tools).

Lab Objectives

By the end of this lab you will be able to:

- Intentionally trigger and recover from 10+ distinct error classes on your 2015 MacBook Pro.
- Read logs with `read_file` and terminal commands to progress `actually_read_the_logs`.
- Generate port conflicts by spawning duplicate servers to unlock `port_3000_taken`.
- Trigger permission-denied events with `chmod` and `sudo` to advance `permission_denied_any_percent`.
- Create dependency installation failures and successes to satisfy `dependency_hell_tourist`.
- Force YAML syntax errors by editing config files to feed `yaml_colon_incident`.
- Collide Docker container names to increment `docker_name_collision`.
- Missing env var errors to progress `forgot_the_env_var`.
- Restart the agent after an error spike to unlock `the_fix_was_restarting`.

Why This Lab Matters

Debugging is not a detour — it is the curriculum. Every stack trace is a lesson. Every port conflict is a networking primer. Every YAML syntax error is a reminder that colons are treacherous. This lab transforms failure from frustration into progress. The `red_text_connoisseur` badge turns your terminal into a badge wall.

Step-by-Step Instructions

Step 1: Setup a Safe Error Sandbox (5 minutes)

Create a disposable folder so you do not damage real work:

```
mkdir -p ~/hermes_error_sandbox && cd ~/hermes_error_sandbox
```

All error-provoking actions happen here.

Step 2: Generate Permission Denied Errors (10 minutes)

Inside the chat, type:

```
terminal: {"command": "touch /root/forbidden.txt"}
```

Expected: Permission denied . Repeat with:

```
terminal: {"command": "sudo ls /var/root"}
```

If it asks for a password, press Ctrl+C to abort — the permission denial still counts.

Repeat 10 times. Check the badge with:

```
curl -s http://127.0.0.1:9119/api/plugins/hermes-achievements/achievements | jq '.achievements[] | select'
```

Troubleshooting: If `sudo` succeeds because your user has passwordless sudo, run `chmod 000 ~/hermes_error_sandbox/locked_file` then try to read it.

Step 3: Spawn Port 3000 Conflicts (10 minutes)

Terminal commands:

```
nohup python3 -m http.server 3000 &
echo $!
```

Now try to start another server on the same port:

```
python3 -m http.server 3000
```

Expected: `OSError: [Errno 48] Address already in use` . Repeat the collision 5 times by killing and restarting the first server.

Inside chat, also run:

```
terminal: {"command": "python3 -m http.server 3000"}
```

This logs the conflict in Hermes' state.

Hardware note: On a 2015 MacBook Pro, running two Python `http.server` processes on port 3000 is harmless. CPU stays under 5%.

Step 4: Trigger Docker Name Collisions (10 minutes)

If Docker is installed:

```
docker run --name hermes_test -d alpine sleep 30
docker run --name hermes_test -d alpine sleep 30
```

Expected: Conflict. The container name `"/hermes_test"` is already in use . Repeat with different names:

```
for i in {1..5}; do docker run --name hermes_dup_$i -d alpine sleep 5; docker run --name hermes_dup_$i -
```

If Docker is not installed, use `terminal` with `docker` anyway — the "command not found" may count as a related error but not toward `docker_name_collision`. Better to skip if Docker is absent.

Step 5: Create YAML Syntax Errors (10 minutes)

Create a broken config:

```
cat > ~/hermes_error_sandbox/broken.yaml << 'EOF'
name: test
  indented_wrong: true
bad:colon:here
EOF
```

Try to parse it:

```
python3 -c "import yaml; yaml.safe_load(open('/Users/$USER/hermes_error_sandbox/broken.yaml'))"
```

Expected: `yaml.scanner.ScannerError` or similar. Repeat 5+ times by introducing different colon misplacements, missing quotes, and unclosed brackets.

Inside chat, also use `read_file` on broken YAMLs — some scanners count doc reads toward `docs_archaeologist` too.

Step 6: Trigger Env Var Errors (10 minutes)

Unset a required variable and try to use it:

```
unset NONEXISTENT_API_KEY
curl -H "Authorization: $NONEXISTENT_API_KEY" https://httpbin.org/get
```

Expected: empty header, but the point is the *absence* of the variable. In Hermes, create a script that tries to load `.env` values that do not exist:

```
cat > ~/hermes_error_sandbox/env_loader.py << 'EOF'
import os
for key in ["MISSING_VAR_1", "MISSING_VAR_2", "MISSING_VAR_3"]:
    val = os.getenv(key)
    if val is None:
        raise EnvironmentError(f"{key} is not set")
EOF
python3 ~/hermes_error_sandbox/env_loader.py
```

Each run throws an `EnvironmentError`. Run 10+ times.

Step 7: Dependency Hell — Failures + Successes (10 minutes)

Install a real package (success):

```
pip install requests
```

Try to install a nonexistent package (failure):

```
pip install fakepackage_that_does_not_exist_12345
```

Repeat. The multi-condition `dependency_hell_tourist` requires 25 install errors + 10 install successes lifetime. If you are far from that, run both commands in a loop:

```
for i in {1..15}; do pip install requests; pip install totally_fake_pkg_$i; done
```

Warning: On a 2015 MacBook, pip installs can spike CPU. Run in batches of 5.

Step 8: Read Logs Like a Detective (10 minutes)

Read Hermes' own logs:

```
read_file: "/Users/thanit/.hermes/logs/latest.log"
```

Also read system logs:

```
terminal: {"command": "log show --last 1h | grep hermes | tail -50"}
```

Each `read_file` or `terminal` log read counts toward `actually_read_the_logs`.

Step 9: Restart After Error Spike (5 minutes)

After all the chaos above, type:

```
/quit
```

Then restart Hermes:

```
hermes --tui
```

If the session had 50+ errors and 4000+ total errors lifetime, `the_fix_was_restarting` may trigger.

Expected Outcome

- 10+ permission denied events logged.
- 5+ port 3000 conflicts recorded.
- 5+ Docker name collisions (if Docker available).
- 5+ YAML syntax errors thrown.
- 10+ env var missing errors logged.
- 5+ pip install failures + successes.

- Logs read at least 3 times via file + terminal.
- Agent restarted successfully after the chaos session.

Lab 6 Use Case Completed

You turned your MacBook into a controlled demolition site. Every error was intentional, every crash was recorded, and every restart was a badge unlocked. The scanner now sees a richer error history.

Training Idea / Self-Improvement Focus

- Errors are not just tracked for badges — they feed the Honcho user model. The agent learns your most common failure modes and can pre-empt them.
- The Curator may create a "common_fixes" skill after reviewing repeated error patterns.
- On a 2015 MacBook, frequent restarts after errors also exercise the SSD. Keep `/tmp` and `~/hermes/logs/` pruned.

Challenge (Recommended)

1. Write a Python script that intentionally triggers all 5 error types in 30 seconds. Run it from Hermes with `execute_code`.
2. Create a `~/hermes/error_journal.md` where you log each error, its cause, and the fix. Feed this to the memory tool.
3. Try to unlock `Port 3000 Is Taken Silver (40)` in one afternoon by scripting server restarts.
4. Read `/var/log/system.log` with `terminal(background=True)` and poll it for background process progress.

Emotional Reward

🏆 You are no longer afraid of red text. Stack traces are souvenirs. Logs are literature. Your Terminal is now a gallery of hard-won badges.

Chapter 6: The Error & Recovery Pipeline — How Hermes Survives Chaos

Top Ideal Study Objectives:

- Trace an error from the tool layer to the achievement scanner.
- Understand why `red_text_connoisseur` and `stack_trace_sommelier` track different events.
- See how port conflicts, permission denials, and YAML errors are categorized.
- Learn the restart detection heuristic for `the_fix_was_restarting`.

Description: This chapter is the architect's map of the debugging subsystem. Lab 6 was the demolition derby. Here we inspect the engine: how errors are caught, logged, counted, and turned into badges. If debugging is detective work, this chapter is the forensics lab.

Actions List (Topics):

1. Locate the error classification map in the source tree.
2. Read the achievement scanner logic that increments lifetime error counters.
3. Trace how `terminal` output is parsed for port/permission/YAML/env patterns.
4. Find the restart detection hook.

Full Source Code Snippets with Line-by-Line Explanations:

```
# error_recovery.py -- Educational pseudocode for error tracking & recovery
# Version: v0.15.1+
```

```

"""
Error & Recovery orchestrator.

Handles:
- Tool-level exception capture
- Terminal output regex matching for known error signatures
- Achievement scanner increment logic
- Restart-after-error heuristics
- Log file reads (for actually_read_the_logs)

All error data is written locally to state.json so a 2015 MacBook
with no cloud connection still earns badges.
"""

import re # Line ~15 -- signature matching in terminal output
import json # Line ~16 -- state.json read/write
import os # Line ~17 -- env var access for env-var errors
from pathlib import Path # Line ~18 -- log file paths

# Line-by-line:
# re : Every terminal output is regex-scanned for:
#     "Permission denied", "Address already in use",
#     "yaml.scanner.ScannerError", "docker: Error",
#     "KeyError", "NameError", etc.
# json : Badge progress is stored in state.json as a flat dict
#       keyed by achievement ID.
# os : When `os.getenv` returns None and the model references
#       the missing var, the env-var error counter increments.
# Path : Log files live in ~/.hermes/logs/*.log. The scanner
#        reads them incrementally, tail-style.

# ---- Terminal output classification ----
# Called after every terminal(background=False) or background process
# stdout/stderr read.

ERROR_SIGNATURES = { # Line ~34
    "port_conflict": re.compile(r"Address already in use.*:(\d+)"), # Line ~35
    "permission_denied": re.compile(r"Permission denied"), # Line ~36
    "yaml_error": re.compile(r"yaml\. (scanner|parser)\."), # Line ~37
    "docker_collision": re.compile(r"container name .* is already in use"), # Line ~38
    "env_var_missing": re.compile(r"EnvironmentError|KeyError.*env"), # Line ~39
    "stack_trace": re.compile(r"Traceback \ (most recent call last\)"), # Line ~40
    "general_error": re.compile(r"Error:|ERROR|error:"), # Line ~41
}

# Line ~35 : Captures port numbers. "Port 3000 Is Taken" looks for
#             port 3000 specifically but also counts general conflicts.
# Line ~40 : Stack traces are distinct from general errors. A single
#             Python exception produces one traceback event but may
#             also increment general_error if the word "Error:" appears.
# Line ~41 : Broad catch-all. This is where `red_text_connoisseur`
#             gets most of its count - every red line in Terminal.

def classify_terminal_output(text: str) -> dict: # Line ~44
    counts = {k: 0 for k in ERROR_SIGNATURES} # Line ~45
    for category, pattern in ERROR_SIGNATURES.items(): # Line ~46
        matches = pattern.findall(text) # Line ~47
        counts[category] = len(matches) # Line ~48
    return counts # Line ~49

# Line ~48 : findall returns a list of matches. Multiple errors in
#             one terminal output are counted separately.

# ---- Achievement state update ----
# Reads state.json, increments counters, writes back.

```

```

def update_error_badge_counts(new_counts: dict) -> None: # Line ~54
    state_path = Path("~/hermes/state.json").expanduser() # Line ~55
    state = json.loads(state_path.read_text()) if state_path.exists() else {} # Line ~56
    badges = state.get("achievements", {}) # Line ~57
    badges.setdefault("red_text_connoisseur", 0) # Line ~58
    badges["red_text_connoisseur"] += sum(new_counts.values()) # Line ~59
    badges.setdefault("stack_trace_sommelier", 0) # Line ~60
    badges["stack_trace_sommelier"] += new_counts.get("stack_trace", 0) # Line ~61
    badges.setdefault("port_3000_taken", 0) # Line ~62
    if str(3000) in str(new_counts.get("port_conflict", "")): # Line ~63
        badges["port_3000_taken"] += new_counts["port_conflict"] # Line ~64
    state_path.write_text(json.dumps(state, indent=2)) # Line ~65

# Line ~59 : red_text_connoisseur is the sum of ALL error categories.
#           This is why it is the easiest badge to unlock naturally.
# Line ~63 : port_3000_taken is gated – it only counts conflicts on
#           port 3000 explicitly, not arbitrary ports.
# Line ~65 : Writes back compact JSON. On a 2015 MacBook SSD this
#           is <1ms even for a 500KB state file.

# ---- Restart-after-error heuristic ----
# Called at session start. If the last session ended with errors
# and the uptime delta is small, assume a restart fix.

def detect_restart_fix(current_session_errors: int, last_session_errors: int, uptime_seconds: float) ->
    if uptime_seconds < 60 and current_session_errors > 0: # Line ~72
        if last_session_errors > 4000: # Line ~73
            if current_session_errors <= 50: # Line ~74
                return True # Line ~75
    return False # Line ~76

# Line ~72 : Uptime < 60s implies a very recent restart.
# Line ~73 : The total error history must be >= 4000 (lifetime).
# Line ~74 : The current session must have <= 50 errors after
#           restart – otherwise it was not a fix, it is still broken.
# Line ~75 : Triggers `the_fix_was_restarting` if both conditions
#           and the multi_condition threshold are met.

```

Master Prompt for This Chapter:

You are continuing my Hermes Agent deep-dive education from Chapter 6. Explain the error and recovery pipeline:

1. How does `classify_terminal_output` distinguish a port conflict from a permission denied? Why are separate regexes used?
2. Why is `red_text_connoisseur` the sum of all error category counts, while `stack_trace_sommelier` only counts Traceback lines?
3. What prevents the achievement state file from corrupting if Hermes crashes while writing `state.json` on a 2015 MacBook?
4. How does `detect_restart_fix` know that a restart was "the fix" rather than just a reboot into the same broken state?
5. Which 5 badges in this unit are Secret and what is the minimal action required to reveal each one?

Conclude with exactly 5 questions to test my understanding.

Chapter Conclusion — What You Have Learned

1. Error classification is regex-based terminal scanning. Every tool call that touches the shell is analyzed for known signatures.
2. `red_text_connoisseur` is intentionally broad — it counts every red line. `stack_trace_sommelier` is narrow — it counts only Traceback blocks.
3. State updates are atomic writes to local JSON. Even if the system crashes mid-write, the old `state.json` is preserved because `write_text` creates a temp file and renames it.

4. Restart detection is heuristic, not causal. It assumes correlation: recent restart + low new errors = fix. It cannot prove causation.
5. Secret badges (Port 3000 Is Taken, Permission Denied Any%, Forgot The Env Var, YAML Colon Incident, Docker Name Collision) are invisible until the first trigger event fires.

5 Questions to Check Your Understanding:

1. You run a terminal command that prints both `Permission denied` and a Python `Traceback`. How many badges increment and by how much?
 2. If `state.json` grows to 2MB because of a huge error history, why does the badge scanner still load in <1s on a 2015 MacBook SSD?
 3. The `port_3000_taken` regex captures `(\d+)` for the port number, but the badge only increments for port 3000. Why capture the digit at all?
 4. You restart Hermes 10 times in a row, but each new session immediately throws 100 errors. Does `the_fix_was_restarting` unlock? Why or why not?
 5. You want to read Hermes logs programmatically to speed up `actually_read_the_logs`. Write the exact Python one-liner to count lines in `~/hermes/logs/latest.log` that contain the word `ERROR`.
-

Unit 7: Session Quality — Lifestyle Patterns, Vibe Coding, and Marathon Endurance

Lab 7: Unlocking Weekend Warrior, Night Owl, Early Bird Cron, Supposed to Be Quick, One More Small Change, Vibe Architect, and Marathon Runner

Level: Intermediate Estimated Time: ~~90 minutes~~ Prerequisites: Hermes Agent v0.15.1 installed, dashboard running at ~~http://127.0.0.1:9119~~, at least 10 prior sessions in `~/hermes/state.db`, and a 2015 MacBook Pro ready for extended thermal loads.

Lab Objectives

By the end of this lab you will be able to:

1. Query your session-quality badge progress via the Achievements API.
2. Design a weekend session strategy to advance **Weekend Warrior**.
3. Execute a late-night spiral that unlocks **Night Owl** and **This Was Supposed To Be Quick**.
4. Schedule a morning cron job that contributes to **Early Bird Cron**.
5. Drive a refactoring loop that advances **One More Small Change** and **Vibe Architect**.
6. Sustain an active session long enough to make progress on **Marathon Runner**.
7. Interpret the difference between `lifetime` and `best_session` tracking kinds.

Why This Lab Matters

Session-quality badges measure *when*, *how long*, and *how deep* you work — not just the raw volume of commands. On a 2015 MacBook Pro, long sessions teach real hardware discipline: thermal throttling after 90 minutes, battery sag below 20 %, and the fan curve that tells you it is time to save a checkpoint. These badges turn your machine's limitations into a meta-game. Understanding them means understanding your own cadence as an operator.

Step-by-Step Instructions

Step 1: Inspect Current Progress (5 minutes)

```
curl -s http://127.0.0.1:9119/api/plugins/hermes-achievements/achievements \
  | jq '.achievements[] | select(.id | IN("weekend_warrior","night_owl","early_bird_cron","supposed_to_k
```

Note the current tier and progress for each target badge. If the API is offline, open `hermes dashboard` → Achievements tab.

Step 2: Weekend Warrior — Run a Saturday Deep Session (15 minutes)

Open Hermes on a Saturday or Sunday. Run:

```
hermes --tui
```

Inside the session, perform at least 50 tool calls (file edits, web searches, terminal commands). Then save:

```
/save
```

Exit. The session is flagged as a weekend session. Thresholds: **Copper 10**, Silver 30, Gold 80, Diamond 200, Olympian 500. Weekend sessions are `lifetime` cumulative.

Step 3: Night Owl — Stay Active After 22:00 (15 minutes)

Start a session between 22:00 and 05:00 local time. On your MacBook:

```
hermes --tui
```

Run a task that naturally extends: refactoring a Python module, scraping a site, or writing a skill. Use at least 30 tool calls before saving. Thresholds: **Copper 20**, Silver 60, Gold 150, Diamond 400, Olympian 1,000. These are also `lifetime` cumulative.

Step 4: Early Bird Cron — Schedule a 06:00 Job (10 minutes)

```
hermes cron add "early bird status check" --schedule "0 6 * * *"
```

Action payload:

```
At 6 AM every day:
1. Run `uptime` and `df -h` via terminal.
2. Save the output to ~/macbook-dailies/$(date +%F).md.
```

Test it once manually:

```
hermes cron run "early bird status check"
```

Thresholds: **Copper 5**, Silver 15, Gold 40, Diamond 100, Olympian 250. Only cron jobs firing between 05:00–09:00 count.

Step 5: Supposed To Be Quick — Let a Task Spiral (20 minutes)

Start a session with a "small" request:

```
Just fix this one CSS spacing issue.
```

Then keep asking follow-ups: test it, tweak it, test again, add a breakpoint, check logs, refactor the component. Do not batch your requests — send each as a separate message. Aim for 60+ messages in one session. Thresholds: **Copper 300**, Silver 600, Gold 1,200, Diamond 2,500, Olympian 6,000. This is a `best_session` peak — only your highest count ever matters.

Step 6: One More Small Change + Vibe Architect — File Iteration Gauntlet (20 minutes)

In the same session (or a new one), repeatedly edit the same file:

```
Patch style.css, then test, then patch again.
```

Each `patch` or `write_file` call counts as a file tool call. Aim for 50+ file tool calls. Separately, create or touch 20+ distinct files (temp configs, stubs, logs). Thresholds:

- **One More Small Change:** Copper 150, Silver 400, Gold 1,000, Diamond 3,000, Olympian 8,000 (`best_session`).
- **Vibe Architect:** Copper 300, Silver 700, Gold 1,500, Diamond 4,000, Olympian 10,000 (`best_session`).

Save and exit.

Step 7: Marathon Runner — Keep the Session Alive (5 minutes)

`best_session` duration is measured in minutes. To make progress, the session must remain active without long idle gaps. On a 2015 MacBook Pro, keep the terminal window open and send a message every few minutes. Thresholds: **Copper 60**, Silver 180, Gold 480, Diamond 1,200, Olympian 3,000 minutes. For this lab, reaching Copper (1 hour) in a single contiguous session is sufficient.

Step 8: Verify Unlocks and Rescan (5 minutes)

```
curl -X POST http://127.0.0.1:9119/api/plugins/hermes-achievements/rescan
```

Re-run the Step 1 query and confirm progress bars moved.

Troubleshooting

- **Badges not updating after a session:** The scan engine caches snapshots. Force a rescan with the POST command above. First-time scans are background and non-blocking.
- **MacBook sleeps mid-session:** macOS sleep pauses the process timer. Disable sleep temporarily with `caffeinate -d` in a second Terminal tab.
- **Cron job not counted for Early Bird:** Verify the system timezone matches the cron schedule. Check with `date` and `hermes cron list`.
- **Vibe Architect not advancing:** Only `write_file`, `patch`, and sometimes `terminal` file redirects count. Verify via dashboard detail view.

Expected Outcome

- Weekend Warrior progress moves from current → current + 1 (or unlocks Copper if this was your 10th weekend session).
- Night Owl and/or Early Bird Cron show visible progress in the dashboard.
- At least one `best_session` badge (Supposed To Be Quick, One More Small Change, Vibe Architect, or Marathon Runner) shows a new personal best.
- You understand that `lifetime` badges reward persistence while `best_session` badges reward peak performance.

Lab 7 Use Case Completed

You now track your own coding lifestyle through Hermes. You know whether you are a weekend sprinter, a midnight debugger, or a morning cron architect. Your 2015 MacBook has survived the gauntlet — and you have the progress bars to prove it.

Training Idea / Self-Improvement Focus

Session quality data is feedback about *you*, not just the machine. If Night Owl is your highest badge, consider whether late-night work is sustainable. If Marathon Runner spikes past Gold, ask whether those 8-hour sessions were productive or just hard to kill. Use the badge data to calibrate your schedule. On limited hardware, intentional pacing beats brute-force endurance.

Challenge (Recommended)

1. Within the next 30 days, aim for **Gold tier** on one `best_session` badge.
2. Keep a Markdown journal at `~/session-quality-log.md`. After each long session, record: date, duration, peak message count, and whether the MacBook thermal-throttled.
3. At the end of 30 days, analyze the log. Do your highest-badge days correlate with better outputs or just longer inputs?

Emotional Reward

You have turned your calendar and your stamina into a scoreboard. That is not vanity — it is telemetry. Outstanding work!

Chapter 7: How Session Quality Is Measured Inside the Scanning Engine

Top Ideal Study Objectives:

- Trace how `state.db` session timestamps are bucketed into weekend, night, and morning categories.
- Understand the difference between `lifetime` accumulation and `best_session` peak detection.
- Connect the scanning engine's incremental cache to the dashboard progress bars you observed in Lab 7.

Description: This chapter examines the backend logic that converts raw session history into the seven Session Quality badges. When the cron job fired at 6 AM in Lab 7, the scanner did not just increment a counter — it parsed the session's ISO timestamp, extracted the weekday and hour, and routed the increment to the correct `lifetime` bucket. For your 60-message spiral session, it compared the new peak to the previous best and atomically updated `best_session` if the new value was higher. On a 2015 MacBook with a spinning disk or old SSD, this scan is cached so that repeated dashboard visits feel instant.

Actions List (Topics):

1. Locate the timestamp parser in the achievements plugin source.
2. Identify the `best_session` comparison logic and its atomic update path.
3. Trace how weekend/night/morning hour filtering is implemented.
4. Observe how the snapshot cache (`SNAPSHOT_TTL_SECONDS = 120`) avoids re-parsing unchanged sessions.

Full Source Code Snippets with Line-by-Line Explanations:

```
# hermes-achievements plugin: scan_engine.py (conceptual reconstruction)

class SessionQualityScanner:
    def __init__(self, state_db_path: str, user_state: dict):
        self.db = state_db_path
        self.state = user_state          # state.json in memory
        self.THRESHOLDS = {
            "weekend_warrior": [10, 30, 80, 200, 500],
            "night_owl": [20, 60, 150, 400, 1000],
            "early_bird_cron": [5, 15, 40, 100, 250],
            "supposed_to_be_quick": [300, 600, 1200, 2500, 6000],
            "one_more_small_change": [150, 400, 1000, 3000, 8000],
            "vibe_architect": [300, 700, 1500, 4000, 10000],
            "marathon_runner": [60, 180, 480, 1200, 3000],
        }

    def scan_session(self, session: dict) -> dict:
        """
        session: {
            "id": "uuid",
```

```

        "started_at": "2026-06-06T23:15:00Z",
        "ended_at": "2026-06-07T01:45:00Z",
        "message_count": 340,
        "file_tool_calls": 180,
        "files_touched": 420,
        "source": "cli"
    }
    """
    dt = datetime.fromisoformat(session["started_at"].replace("Z", "+00:00"))
    duration_min = self._duration(session)

    # --- Lifetime bucketing by timestamp ---
    if dt.weekday() >= 5:                                # Line ~45
        self.state["weekend_warrior"] += 1              # Line ~46

    if 22 <= dt.hour or dt.hour < 5:                    # Line ~49
        self.state["night_owl"] += 1                   # Line ~50

    # Note: early_bird_cron is handled by the cron dispatcher,
    # not the session scanner, because it tracks cron executions.

    # --- Best-session peak detection ---
    self._maybe_update_best("supposed_to_be_quick", session["message_count"]) # Line ~57
    self._maybe_update_best("one_more_small_change", session["file_tool_calls"]) # Line ~58
    self._maybe_update_best("vibe_architect", session["files_touched"]) # Line ~59
    self._maybe_update_best("marathon_runner", duration_min) # Line ~60

    return self.state

def _maybe_update_best(self, badge_id: str, new_value: int):
    """
    Atomic peak update. Only replaces the stored best if new_value is strictly greater.
    This is a single dict assignment – thread-safe because Hermes is single-process
    and the GIL protects the dict mutation.
    """
    old_best = self.state.get(f"{badge_id}_best", 0) # Line ~70
    if new_value > old_best:                          # Line ~71
        self.state[f"{badge_id}_best"] = new_value    # Line ~72
        print(f"New best for {badge_id}: {new_value}") # Line ~73

def _duration(self, session: dict) -> int:
    start = datetime.fromisoformat(session["started_at"].replace("Z", "+00:00"))
    end = datetime.fromisoformat(session["ended_at"].replace("Z", "+00:00"))
    return int((end - start).total_seconds() // 60) # Line ~79

# Line-by-line:
# Line 45-46 : Weekend detection uses Python's datetime.weekday().
#             Monday=0 ... Saturday=5, Sunday=6. Sessions starting on
#             Saturday or Sunday increment Weekend Warrior.
#
# Line 49-50 : Night Owl uses hour-of-day filtering. 22:00 to 04:59 inclusive.
#             A session starting at 23:15 counts. A session starting at 05:00
#             does not. This is a simple integer range check – fast on old
#             hardware because it never touches the session body.
#
# Line 57-60 : Peak detection runs for every session. Even if you had a quiet
#             10-message session, the scanner still checks. The dict lookup is
#             O(1) and the comparison is a single CPU instruction. On a 2015
#             MacBook, scanning 1,000 sessions takes < 100 ms after caching.
#
# Line 70-73 : The old best is read, compared, and overwritten only if the
#             new value is strictly greater. This prevents regressions.
#             The print line becomes a dashboard event in production.
#
# Line 79 : Duration is integer minutes, truncated. A session of 59 minutes
#          and 59 seconds contributes 59, not 60, to Marathon Runner.

```

Master Prompt for This Chapter:

You are continuing my Hermes Agent deep-dive education from Chapter 7.

Explain the Session Quality scanning engine:

- How does the scanner distinguish a weekend session from a weekday session using only the `started_at` timestamp (Line 45)?
- Why is Night Owl defined as 22:00-05:00 instead of a simpler "after midnight" rule?
- What is the computational cost of `_maybe_update_best()` on a 2015 MacBook Pro with 1,000 historical sessions?
- If I run two sessions on the same Saturday, how many times does Weekend Warrior increment? What kind metric drives this?
- Marathon Runner uses `duration_min`. If my session starts at 23:50 and ends at 00:40 the next day, what value is recorded and why?
- Why does Early Bird Cron track cron job executions rather than regular session starts, and where is that counted instead?

Use a 2015 MacBook Pro as the hardware context.

Conclude with exactly 5 questions to test my understanding.

Chapter Conclusion — What You Have Learned

1. `lifetime` badges count events cumulatively. Weekend Warrior and Night Owl never reset; every qualifying session adds +1.
2. `best_session` badges store a single peak. If you hit 340 messages once, that number persists until you beat it.
3. Timestamp parsing is lightweight. The scanner only reads the session header (`started_at`, `ended_at`) for quality badges, not the full message log.
4. Atomic comparison protects progress. The `>` check in `_maybe_update_best` guarantees that a bad session cannot lower your badge.
5. Caching makes repeated dashboard visits instant. After the first scan, `SNAPSHOT_TTL_SECONDS = 120` reuses the payload without re-parsing `state.db`.

5 Questions to Check Your Understanding:

1. You start a session on Friday at 21:00 and finish at 23:30. Which `lifetime` badge increments, if any? Explain using Line 45 and Line 49.
2. If your current best for `supposed_to_be_quick` is 620 (Silver tier) and today's session reaches 1,100 messages, what tier do you unlock? Which line in the scanner makes this decision?
3. Why is `_maybe_update_best` considered atomic in the Hermes single-process model, and what would change if Hermes moved to a multi-process architecture?
4. A cron job is scheduled for 08:30 every day. Does it count toward Early Bird Cron? Why or why not?
5. Your 2015 MacBook takes 2.3 seconds to scan `state.db` the first time, but 80 ms on subsequent visits. Which caching mechanism is responsible, and what is the TTL?

Unit 8: Social & Community — Shared Session, Team Delegate, and Cross Platform

Lab 8: Sharing Sessions, Delegating to Multi-User Subagents, and Using CLI + Gateway Together

Level: Intermediate Estimated Time: ~75 minutes Prerequisites: Hermes Agent v0.15.1, dashboard running, at least one gateway configured (Telegram or Discord from earlier labs), and 5+ completed sessions eligible for sharing.

Lab Objectives

By the end of this lab you will be able to:

1. Export a session transcript to a shareable Markdown file using `/save` and manual export.
2. Identify which sessions qualify as "shared" for the **Shared Session** badge counter.
3. Spawn a multi-user `delegate_task` that routes to another Hermes profile or user.
4. Verify that a **Team Delegate** call increments the lifetime counter.
5. Use the CLI and a gateway (Telegram) in the same workflow to advance **Cross Platform**.
6. Understand the API endpoints for session export and recent-unlock notifications.
7. Generate a PNG share card (1200×630) for social media from an unlocked badge.

Why This Lab Matters

Hermes is not a solo tool. The Social & Community badges measure how well you integrate the agent into collaborative workflows. **Shared Session** surfaces work to teammates. **Team Delegate** distributes load across agents. **Cross Platform** ensures you are not locked into a single interface. On a 2015 MacBook Pro with 16 GB RAM, memory pressure makes it hard to run multiple local models at once — delegating heavy tasks to a teammate's machine or a remote server via `delegate_task` is a practical necessity, not just a badge hunt.

Step-by-Step Instructions

Step 1: Check Your Current Social Badge Progress (5 minutes)

```
curl -s http://127.0.0.1:9119/api/plugins/hermes-achievements/achievements \
  | jq '.achievements[] | select(.id | IN("shared_session","team_delegate","cross_platform")) | {id, tier}'
```

Thresholds:

- **Shared Session:** Copper 5, Silver 15, Gold 40, Diamond 100, Olympian 250 (`lifetime`)
- **Team Delegate:** Copper 10, Silver 30, Gold 80, Diamond 200, Olympian 500 (`lifetime`)
- **Cross Platform:** Copper 2, Silver 3, Gold 4, Diamond 5, Olympian 6 (`lifetime`)

Note the current tier. For most learners, Shared Session and Cross Platform advance naturally once you understand what counts.

Step 2: Share a Session to File (10 minutes)

Inside a Hermes TUI session:

```
/save
```

This writes the transcript to `~/.hermes/sessions/<uuid>.jsonl`. To turn it into a shareable artifact:

```
mkdir -p ~/shared-hermes-sessions
cp ~/.hermes/sessions/$(ls -t ~/.hermes/sessions/ | head -1) ~/shared-hermes-sessions/
```

Then generate a Markdown summary:

```
Read the most recent session file in ~/shared-hermes-sessions/ and convert it to a readable Markdown summary
```

Sharing methods that count:

- Exporting to PDF via `pandoc`.
- Copy-pasting the `/save` output into a GitHub gist or Slack.
- Using the Hermes dashboard "Export" button (which fires the share event).

Step 3: Share via Dashboard and Generate a PNG Card (10 minutes)

Open `hermes dashboard` → Achievements tab. Click any unlocked badge. If v0.4.0+ is installed, a "Generate Share Card" button appears. Click it. The server renders a 1200×630 PNG using the badge name, tier, and current progress. Save it to:

```
~/Desktop/hermes-badge-share.png
```

This counts as a `shared_session` event in some configurations. Verify by checking the API again after the action.

Step 4: Team Delegate — Spawn a Subagent for Another User (20 minutes)

In Hermes TUI, delegate a task with an explicit target user:

```
delegate_task to user "teammate-profile" with the following task:
"Analyze the CPU temperature logs in ~/macbook-logs/ and return a one-sentence summary."
```

The exact syntax varies by client, but the principle is the same: the `delegate_task` tool receives a `target_user` parameter. If the target user is different from the caller, the counter increments. Thresholds:

- **Team Delegate:** Copper 10, Silver 30, Gold 80, Diamond 200, Olympian 500 (`lifetime`).

If you do not have a teammate profile, simulate it:

```
# test_delegate.py
import json, os

delegate_event = {
    "tool": "delegate_task",
    "args": {"task": "Summarize ~/notes.txt", "target_user": "demo-user"},
    "caller": "hermes-learn"
}

log_path = os.path.expanduser("~/hermes/delegate-test.log")
with open(log_path, "a") as f:
```



```
f.write(json.dumps(delegate_event) + "\n")
print("Simulated team delegate logged.")
```

Run it:

```
python3 ~/test_delegate.py
```

Then force a rescan:

```
curl -X POST http://127.0.0.1:9119/api/plugins/hermes-achievements/rescan
```

Step 5: Cross Platform — Use CLI and Telegram in One Session (20 minutes)

Prerequisite: Telegram gateway running (see Lab 6).

1. Start a CLI session:

```
hermes --tui
```

2. Inside the session, send a message to your Telegram bot:

Use the telegram gateway to send "Cross-platform test initiated from CLI" to my chat.

3. Receive a reply on Telegram. Reply from Telegram.
4. Back in the CLI session, check if the Telegram reply was picked up:

Did any gateway messages arrive while I was away?

Each distinct platform used within the tracked lifetime period counts toward Cross Platform. Thresholds:

- **Cross Platform:** Copper 2, Silver 3, Gold 4, Diamond 5, Olympian 6 (`lifetime`).

If Telegram is unavailable, substitute Discord or the web dashboard message interface.

Step 6: Monitor Recent Unlocks via API (10 minutes)

```
curl -s http://127.0.0.1:9119/api/plugins/hermes-achievements/recent-unlocks \
  | jq '.[0] | {name, tier, unlocked_at}'
```

This endpoint returns the last 10 unlocked badges with ISO-8601 timestamps. If a social badge unlocked during this lab, it will appear here.

Troubleshooting

- **shared_session not incrementing:** The old `/save` command alone does not always trigger the share counter. Use the dashboard "Export" button or share via a gateway message for reliable counting.

- **Team Delegate requires distinct users:** Delegating to yourself or to an anonymous subagent does not increment the badge. Verify that `target_user` is present and different from the caller.
- **Cross Platform counts platforms, not sessions:** Using CLI five times and Telegram once counts as 2 platforms, not 6. You need 6 *distinct* platforms for Olympian.
- **PNG share cards not rendering:** Ensure Hermes is v0.4.0+. The card generator uses headless Chrome; if Chrome is missing on your MacBook, install it via MacPorts: `sudo port install chromium`.

Expected Outcome

- Shared Session progress increments by at least +1.
- Team Delegate progress increments by at least +1 (or is simulated and rescanned).
- Cross Platform progress shows at least 2 platforms (CLI + one gateway).
- You understand that social badges reward *integration* into teams and workflows, not raw volume.

Lab 8 Use Case Completed

You have shared a session artifact, simulated a team delegation, and bridged CLI and gateway usage. Your Hermes agent is no longer a local-only tool — it is a node in a collaborative network. Even on a 2015 MacBook, delegation lets you offload heavy work to remote agents.

Training Idea / Self-Improvement Focus

Social badges are the most "invisible" category because they depend on external people and systems. Set a weekly reminder: share one Hermes session summary with a colleague. After 15 weeks, you hit Silver on Shared Session. After 250 shares, you hit Olympian — and you will have built a library of reusable session artifacts. The badge is a side effect of a healthy documentation habit.

Challenge (Recommended)

1. Create a cron job that runs every Monday at 9 AM and automatically shares the previous week's session summary to a Telegram group.
2. The cron job should:
 - Find the 5 most recent sessions in `~/hermes/sessions/`.
 - Summarize them with `write_file`.
 - Send the summary to Telegram.
 - Log the share event so the badge counter increments.
3. Test the cron job manually with `hermes cron run`.

Emotional Reward

You have made Hermes a team player. Delegation is not laziness — it is architecture. Well done!

Chapter 8: How Social Achievements Are Tracked Across Gateways and Delegates

Top Ideal Study Objectives:

- Trace the `share` event from dashboard click to API call to state.json update.
- Understand how `delegate_task` records distinct target users for Team Delegate.

- Identify the platform registry that maps gateway IDs to Cross Platform counters.
- Connect the PNG share card renderer to the social badge pipeline.

Description: This chapter examines the backend mechanisms behind the three Social & Community badges. Unlike session-quality badges, which inspect timestamps, social badges inspect *metadata*: who the session was shared with, who the delegate target was, and which gateway IDs appeared in the user's lifetime history. On a 2015 MacBook Pro with limited uplink bandwidth, the PNG card renderer is a local headless-Chrome job — it does not upload to a cloud. Sharing is a local file operation that *logs* an event, which the scanner later counts.

Actions List (Topics):

1. Locate the share event logger in the dashboard frontend or plugin API.
2. Identify the `delegate_task` metadata parser that extracts `target_user`.
3. Trace the platform registry (CLI, Telegram, Discord, Web Dashboard, Email, SMS).
4. Observe how the PNG card generator consumes badge state and produces a 1200×630 image.

Full Source Code Snippets with Line-by-Line Explanations:

```
# hermes-achievements plugin: social_tracker.py (conceptual reconstruction)

class SocialTracker:
    PLATFORMS = {"cli", "telegram", "discord", "web_dashboard", "email", "sms"}

    def __init__(self, user_state: dict):
        self.state = user_state

    def log_share(self, session_id: str, method: str):
        """
        Called by the dashboard "Export" button or `/share` command.
        method: "dashboard_export", "gateway_send", "gist_upload", "pdf_render"
        """
        event = {
            "session_id": session_id,
            "method": method,
            "shared_at": datetime.utcnow().isoformat() + "Z"
        }
        self.state.setdefault("shared_sessions", []).append(event) # Line ~22
        self.state["shared_session_count"] = len(self.state["shared_sessions"]) # Line ~23
        print(f"[Social] Share logged. Total: {self.state['shared_session_count']}") # Line ~24

    def log_delegate(self, task_id: str, caller: str, target: str):
        """
        Called by the delegate_task tool wrapper.
        Only increments if caller != target.
        """
        if caller == target: # Line ~31
            print("[Social] Self-delegate does not count for Team Delegate.") # Line ~32
            return # Line ~33

        event = {
            "task_id": task_id,
            "caller": caller,
            "target": target,
            "delegated_at": datetime.utcnow().isoformat() + "Z"
        }
        self.state.setdefault("delegates", []).append(event) # Line ~41
        self.state["team_delegate_count"] = len(self.state["delegates"]) # Line ~42

    def log_platform_use(self, platform_id: str):
        """
        Called at session start. platform_id must be in PLATFORMS.
        """
```

```

        if platform_id not in self.PLATFORMS:
            print(f"[Social] Unknown platform: {platform_id}")
            return

        used = self.state.setdefault("platforms_used", set())
        used.add(platform_id)
        self.state["platforms_used"] = used
        self.state["cross_platform_count"] = len(used)

# Line-by-line:
# Line 22-24 : The share event is appended to a list and the counter is
#               recomputed from list length. This is slightly expensive for
#               Olympian (250 shares) but trivial on a 2015 MacBook.
#               The list preserves provenance; the counter is a convenience.
#
# Line 31-33 : Self-delegation guard. If you delegate to your own profile,
#               the tool executes but the social tracker skips the increment.
#               This prevents gaming by spawning fake subagents.
#
# Line 41-42 : Legitimate delegates are logged with caller and target.
#               The counter is len(delegates), not len(unique targets).
#               You can delegate to the same teammate 500 times and still
#               reach Olympian.
#
# Line 49-51 : Platform whitelist. Invalid platform IDs (e.g., a misspelled
#               "telegam") are silently ignored. This prevents pollution.
#
# Line 54-57 : A set deduplicates platforms. CLI + Telegram + Discord = 3.
#               Repeating Telegram does not increment. The set is serialized
#               to JSON list in state.json because JSON has no native set.

```

Master Prompt for This Chapter:

You are continuing my Hermes Agent deep-dive education from Chapter 8. Explain the Social & Community tracking pipeline:

- When I click "Export" in the dashboard, which function is called (Line 22), and what data structure stores the share history?
- If I delegate a task to myself, does Team Delegate increment? Which line enforces this rule, and why is it necessary?
- Cross Platform uses a set (Line 54). If I first use CLI, then Telegram, then CLI again, what is the value of cross_platform_count after each step?
- The PNG share card renderer reads state.json. If I delete state.json and force a rescan, will the share card still show my badges? Why or why not?
- How many distinct platform IDs are defined in PLATFORMS, and which two are the easiest for a solo developer to unlock on a 2015 MacBook Pro?

Use a 2015 MacBook Pro as the hardware context. Conclude with exactly 5 questions to test my understanding.

Chapter Conclusion — What You Have Learned

1. Shared Session tracks export events, not file saves. Using the dashboard "Export" button is the most reliable way to increment.
2. Team Delegate requires a distinct caller and target. Self-delegation is explicitly filtered out to prevent badge farming.
3. Cross Platform uses a set, not a counter. Repeated use of the same platform does not help; you must expand to new interfaces.
4. Social badges are metadata-driven. The scanner inspects session headers and tool wrappers, not message content.
5. PNG cards are local renders. They work offline, which is ideal for a 2015 MacBook that may not always have reliable Wi-Fi.

5 Questions to Check Your Understanding:

1. You run `hermes --tui` (CLI) and inside it delegate a task to yourself. Which social badge counters, if any, increment? Explain using Lines 31-33 and 54-57.
 2. If the `shared_sessions` list contains 250 events but `shared_session_count` is 247, what is the most likely cause, and how would you fix it?
 3. You have used CLI, Telegram, Discord, and Web Dashboard. What tier is Cross Platform, and how many more platforms do you need for Olympian?
 4. The PNG share card generator fails with a "Chrome not found" error on your MacBook. What is the exact MacPorts command to fix it, and which version of Hermes introduced PNG cards?
 5. A teammate delegates a task *to* you via `delegate_task`. Does this increment *your* Team Delegate counter, *theirs*, both, or neither? Explain the directionality of the event log.
-

Unit 9: Secret Achievements — The Hidden Badges of Hermes

Lab 9: Unlocking Port 3000 Taken, Permission Denied Any%, Forgot the Env Var, YAML Colon Incident, Docker Name Collision, Rollback Wizard, One Character Fix, and Legend

Level: Advanced Estimated Time: ~120 minutes Prerequisites: Hermes Agent v0.15.1, dashboard accessible, terminal and Docker (optional) available, and permission to trigger intentional errors on a 2015 MacBook Pro without breaking production systems.

Lab Objectives

By the end of this lab you will be able to:

1. Trigger each of the 7 non-Legend secret achievements in a controlled way.
2. Distinguish secret `lifetime` badges from secret `multi_condition` badges.
3. Use the `/rollback` command correctly to advance **Rollback Wizard**.
4. Craft a single-character patch that satisfies the **One Character Fix** multi-condition.
5. Interpret the hidden-until-triggered state of secret badges in the dashboard.
6. Explain why **Legend** is the ultimate endgame badge and outline a realistic path to it.
7. Force a full rescan and verify secret badge discovery in `state.json`.

Why This Lab Matters

Secret achievements are invisible until your behavior triggers the first signal. They document the "scars" of real engineering: port conflicts, permission failures, missing env vars, YAML typos, Docker collisions, panic rollbacks, and microscopic bug fixes. These are not participation trophies — they are forensic evidence that you have fought real systems and won (or at least survived). On a 2015 MacBook Pro with 16 GB RAM, you will hit these errors *naturally* because older hardware has tighter resource constraints. This lab teaches you to trigger them intentionally so you can recognize them when they happen for real.

Step-by-Step Instructions

Step 1: Verify Secret Badge States (5 minutes)

Secret badges are invisible in the dashboard until unlocked. Confirm via API:

```
curl -s http://127.0.0.1:9119/api/plugins/hermes-achievements/achievements \
  | jq '.achievements[] | select(.id | IN("port_3000_taken", "permission_denied_any_percent", "forgot_the_
```

Initially, `visible` should be `false` and `tier` should be `null` for most. If you have already triggered one, it will show `visible: true`.

Step 2: Port 3000 Is Taken — Trigger a Port Conflict (10 minutes)

Start a server on port 3000 in one Terminal tab:

```
python3 -m http.server 3000
```

In a second tab, inside a Hermes session, try to start another server:

```
hermes --tui
```

Then ask:

```
Run a Python HTTP server on port 3000 using terminal.
```

The agent executes:

```
python3 -m http.server 3000
```

It fails with:

```
OSError: [Errno 48] Address already in use
```

This error is logged. The achievements scanner counts it toward **Port 3000 Is Taken**.

Thresholds: **Copper 15**, Silver 40, Gold 100, Diamond 300, Olympian 1,000 (`lifetime`).

Kill the first server (Ctrl+C) before proceeding.

Step 3: Permission Denied Any% — Trigger a `sudo` or File-System Failure (10 minutes)

In a Hermes session, ask:

```
Read the file /root/secret.txt
```

The agent attempts:

```
cat /root/secret.txt
```

It fails with:

```
cat: /root/secret.txt: Permission denied
```

Alternatively, run a Docker command without `sudo` on a Docker socket:

```
Run: docker run --rm hello-world
```

If Docker requires root, the error is logged as "permission denied."

Thresholds: **Copper 25**, Silver 75, Gold 200, Diamond 600, Olympian 1,500 (`lifetime`).

Step 4: Forgot The Env Var — Simulate a Missing Environment Variable (10 minutes)

Create a script that expects a variable:

```
cat > ~/env_test.py << 'EOF'
import os
api_key = os.environ["MY_API_KEY"]
print(f"Key is {api_key}")
EOF
```

In a Hermes session, run it without setting the variable:

Execute the Python script at ~/env_test.py

The agent runs:

```
python3 ~/env_test.py
```

It fails with:

```
KeyError: 'MY_API_KEY'
```

This is an env-var error. Each instance counts toward **Forgot The Env Var**.

Thresholds: **Copper 5,000**, Silver 15,000, Gold 40,000, Diamond 100,000, Olympian 250,000 (`lifetime`).

Step 5: YAML Colon Incident — Introduce a Syntax Error (10 minutes)

Create a malformed YAML file:

```
cat > ~/broken.yaml << 'EOF'
server:
  host: localhost
  port: 3000
  debug: true
name: Test Config
EOF
```

That file is actually valid. Now introduce a colon-without-space error:

```
cat > ~/broken.yaml << 'EOF'
server:
  host: localhost
  port:3000
  debug: true
EOF
```

In a Hermes session, ask the agent to parse it:

Parse the YAML file at ~/broken.yaml using Python and print the port.

The agent will run:


```
import yaml
with open("/Users/thanit/broken.yaml") as f:
    data = yaml.safe_load(f)
    print(data["server"]["port"])
```

It fails with a YAML scanner error. This counts toward **YAML Colon Incident**.

Thresholds: **Copper 1,000**, Silver 3,000, Gold 8,000, Diamond 20,000, Olympian 50,000 (`lifetime`).

Step 6: Docker Name Collision — Run a Container Twice (10 minutes)

If Docker is installed:

```
docker run -d --name test_hermes httpd:alpine
```

Then in a Hermes session:

```
Run a Docker container named test_hermes using the httpd:alpine image.
```

The agent executes:

```
docker run -d --name test_hermes httpd:alpine
```

It fails with:

```
docker: Error response from daemon: Conflict. The container name "/test_hermes" is already in use.
```

This counts. Clean up:

```
docker rm -f test_hermes
```

Thresholds: **Copper 75**, Silver 200, Gold 600, Diamond 1,500, Olympian 4,000 (`lifetime`).

If Docker is unavailable, skip and note that this badge requires Docker workflows.

Step 7: Rollback Wizard — Use `/rollback` After an Error (10 minutes)

In a Hermes session, make a destructive edit:

```
Write "DESTROYED" to ~/rollback_test.txt
```

The agent overwrites the file. Now realize the mistake and execute:

```
/rollback
```

The agent restores the previous state. Each `/rollback` call increments the **Rollback Wizard** counter.

Thresholds: **Copper 500**, Silver 1,500, Gold 4,000, Diamond 10,000, Olympian 25,000 (`lifetime`).

Step 8: One Character Fix — Patch After an Error (15 minutes)

This is a `multi_condition` secret badge. Requirements (from the catalog):

- Make a tiny single-character patch.
- It must occur after at least 5 errors in the session.
- Your total lifetime errors across all sessions must be at least 4,000.

In a Hermes session, first generate 5 trivial errors (e.g., run missing files). Then create a file with a one-character bug:

```
cat > ~/one_char.py << 'EOF'
def greet(name):
    return "Hello, " + name + "!"

print(greet("world"))
EOF
```

Note the missing closing parenthesis on the last line. Ask the agent:

```
Run ~/one_char.py. It should fail. Then fix it with the smallest possible patch.
```

The agent identifies the missing `)` and applies:

```
patch({
  "path": "~/one_char.py",
  "old_string": 'print(greet("world"))',
  "new_string": 'print(greet("world"))'
})
```

This is a single-character addition after 5+ errors. If your lifetime error total is $\geq 4,000$, the badge unlocks.

Step 9: Legend — Understand the Endgame (5 minutes)

Legend requires reaching Olympian tier in ALL other achievements. It is the ultimate `multi_condition` secret. Do not attempt to unlock it today; it is a months-long goal. Verify the conditions:

```
curl -s http://127.0.0.1:9119/api/plugins/hermes-achievements/achievements \
  | jq '[.achievements[] | select(.tier != "olympian") | .id] | length'
```

If the count is 0, you have Legend. If it is 60, you have a long road ahead. Keep a checklist.

Step 10: Rescan and Verify (5 minutes)

```
curl -X POST http://127.0.0.1:9119/api/plugins/hermes-achievements/rescan
curl -s http://127.0.0.1:9119/api/plugins/hermes-achievements/recent-unlocks
```

Troubleshooting

- **Secret badges remain invisible:** They only appear after the first trigger. If you have never seen a port conflict, Permission Denied Any% stays hidden. Trigger it intentionally in this lab.

- **One Character Fix requires 4,000 lifetime errors:** If you are a new user, this badge is locked until you accumulate enough errors naturally. Use Hermes heavily on real projects.
- **Docker Name Collision requires Docker:** If Docker is not installed, this badge is unavailable. Install Docker Desktop for Mac or skip it.
- **Rollback Wizard counts `/rollback` calls:** Using `patch` to undo edits does not count. You must use the explicit `/rollback` command.
- **Legend is not a lifetime badge:** It is evaluated by a bulk check that iterates all achievements. Deleting `state.json` and rescanning does not help — the data comes from `state.db`.

Expected Outcome

- At least 3 secret badges are now visible in the dashboard.
- You understand the trigger conditions for all 7 non-Legend secrets.
- One Character Fix is either unlocked (if error threshold met) or planned for future unlocking.
- You have a realistic timeline for Legend based on your current Olympian count.

Lab 9 Use Case Completed

You have intentionally triggered the hidden badges of Hermes. These are not vanity metrics — they map to real failure modes in software engineering. The next time you see "Port 3000 is taken" at 2 AM, you will recognize it as progress toward Olympian.

Training Idea / Self-Improvement Focus

Secret badges teach resilience. The highest-error badge (Forgot The Env Var, Olympian 250,000) is only reachable if you work at scale. Instead of fearing errors, log them. After every error, ask: "Which badge is this feeding?" On a 2015 MacBook Pro, you will generate errors faster than on modern hardware because resource limits are tighter. Turn your hardware constraints into badge acceleration.

Challenge (Recommended)

1. In the next 14 days, aim to make **YAML Colon Incident** visible (Copper: 1,000 errors is hard to trigger intentionally, but 1 is enough for discovery).
2. Keep an "Error Log" skill:
 - Trigger: "I got an error"
 - Action: log the error type, badge it feeds, and a one-sentence fix.
3. After 14 days, review the error log. Which error type do you generate most often? That is your dominant learning vector.

Emotional Reward

You have looked behind the curtain. The invisible badges are the most honest ones — they reward the work no one else sees. Masterful work!

Chapter 9: How the Scanning Engine Discovers and Unlocks Secret Achievements

Top Ideal Study Objectives:

- Trace the visibility toggle from `visible=false` to `visible=true` on first trigger.
- Understand why secret badges use both `lifetime` and `multi_condition` tracking kinds.

- Map the scanner's error-type classification regexes to the secret badge IDs.
- Examine the Legend evaluation function and its computational cost on a 2015 MacBook Pro.

Description: This chapter reveals the internal mechanics of the 8 secret achievements. Unlike visible badges, secrets start in an undiscovered state. The scanner runs a classification pass over every session's error log, tool calls, and rollback events. When a qualifying signal is detected for the first time, the badge flips from hidden to discovered. On a 2015 MacBook Pro with a SATA SSD or Fusion Drive, the full scan is the slowest operation in the achievement pipeline — which is why vo.3.0 introduced snapshot caching.

Actions List (Topics):

1. Locate the secret badge registry and its default visibility map.
2. Identify the error-classification regexes that detect port conflicts, permission denied, and YAML syntax errors.
3. Trace the `multi_condition` evaluator for One Character Fix and Legend.
4. Observe the snapshotted scan path that skips unchanged sessions.

Full Source Code Snippets with Line-by-Line Explanations:

```
# hermes-achievements plugin: secret_scanner.py (conceptual reconstruction)

class SecretScanner:
    SECRETS = {
        "port_3000_taken": {"kind": "lifetime", "pattern": r"Address already in use.*:3000"},
        "permission_denied_any_percent": {"kind": "lifetime", "pattern": r"Permission denied|operation"},
        "forgot_the_env_var": {"kind": "lifetime", "pattern": r"KeyError: ['\"].*['\"]|environ\[|"},
        "yaml_colon_incident": {"kind": "lifetime", "pattern": r"yaml\. (safe_load|load|dump) |Scanr"},
        "docker_name_collision": {"kind": "lifetime", "pattern": r"Conflict\. The container name.*is"},
        "rollback_wizard": {"kind": "lifetime", "signal": "rollback_command_used"},
        "one_character_fix": {"kind": "multi_condition", "conditions": {
            "tiny_patch": True, "session_errors": 5, "lifetime_errors": 4000
        }},
        "legend": {"kind": "multi_condition", "conditions": {
            "all_olympian": True
        }},
    }

    def __init__(self, user_state: dict):
        self.state = user_state

    def classify_error(self, error_text: str, session_stats: dict):
        """
        Called once per error in the session log.
        """
        for badge_id, meta in self.SECRETS.items():
            if meta["kind"] != "lifetime":
                continue

            pattern = meta.get("pattern")
            if pattern and re.search(pattern, error_text, re.I):
                self._increment_lifetime(badge_id)
                self._ensure_visible(badge_id)

    def _increment_lifetime(self, badge_id: str):
        self.state.setdefault(badge_id, 0)
        self.state[badge_id] += 1
        print(f"[Secret] {badge_id} ++ = {self.state[badge_id]}")

    def _ensure_visible(self, badge_id: str):
        if not self.state.get(f"{badge_id}_visible", False):
            self.state[f"{badge_id}_visible"] = True
            print(f"[Secret] {badge_id} DISCOVERED!")
```

```

def check_multi_condition(self, session_stats: dict, total_errors: int):
    """
    Evaluates One Character Fix and Legend after each session scan.
    """
    # --- One Character Fix ---
    ocf = self.SECRETS["one_character_fix"] # Line ~57
    if session_stats.get("tiny_patch", False) \ # Line ~58
        and session_stats.get("errors_in_session", 0) >= 5 \ # Line ~59
        and total_errors >= 4000: # Line ~60
        self._ensure_visible("one_character_fix") # Line ~61
        # Note: OCF has no lifetime counter; it is a one-time unlock.

    # --- Legend ---
    legend = self.SECRETS["legend"] # Line ~65
    olympian_count = sum( # Line ~66
        1 for b in self.state.get("achievements", []) # Line ~67
        if b.get("tier") == "olympian" # Line ~68
    )
    total_achievements = len(self.state.get("achievements", [])) # Line ~70
    if olympian_count == total_achievements \ # Line ~71
        and total_achievements > 0: # Line ~72
        self._ensure_visible("legend") # Line ~73
        print("[Secret] LEGEND UNLOCKED!") # Line ~74

# Line-by-line:
# Line 29-31 : The scanner iterates all secrets but skips multi_condition
#               entries during per-error classification. Lifetime secrets
#               are checked against every error line; multi_condition secrets
#               are evaluated once per session after stats are aggregated.
#
# Line 34-37 : Regex matching with case-insensitive flag (re.I). The pattern
#               for "port_3000_taken" specifically targets port 3000 because
#               it is the most common dev-server port. Other ports are ignored.
#               This is why the badge is called "Port 3000 Is Taken" exactly.
#
# Line 41-43 : Lifetime counters are simple integers. They grow without bound
#               until they hit the threshold. There is no ceiling; you can
#               exceed Olympian and the counter keeps climbing.
#
# Line 47-49 : Visibility flip. Once discovered, a secret badge is never
#               hidden again. Even if the counter is reset (which requires
#               POST /reset-state), the visibility bit remains unless the
#               entire state.json is wiped.
#
# Line 57-61 : One Character Fix requires a boolean flag (tiny_patch), a
#               session-local error count, and a global error count. The
#               tiny_patch flag is set by the patch tool when it detects
#               an addition or deletion of exactly 1 character.
#
# Line 65-74 : Legend iterates EVERY achievement and checks if tier equals
#               "olympian". On a 2015 MacBook Pro with 60+ achievements,
#               this is a list scan of ~60 dict lookups – negligible cost.
#               But it only runs after a full rescan, not after every session.

```

Master Prompt for This Chapter:

You are continuing my Hermes Agent deep-dive education from Chapter 9.

Explain the secret achievement discovery engine:

- When an error matches the regex in Line 35, what two side effects occur (Lines 36-37)? Why is the increment separate from the visibility flip?
- One Character Fix is multi_condition. Which three conditions must all be true (Lines 58-60), and which tool sets the tiny_patch flag?
- If I have 59 achievements at Olympian and 1 at Diamond, what does Line 71 evaluate to? How many more achievements must reach Olympian before Legend triggers?

```
- The re.search in Line 35 scans every error string. For a user with
  75,000 lifetime errors, how many regex matches execute during a full
  rescan? Is this a performance concern on a 2015 MacBook Pro?
- Why does the scanner skip multi_condition secrets in the per-error loop
  (Line 31), and what ensures they are still evaluated?
Use a 2015 MacBook Pro as the hardware context.
Conclude with exactly 5 questions to test my understanding.
```

Chapter Conclusion — What You Have Learned

1. Secret badges start invisible. The scanner flips visibility on first trigger, and the flip is permanent.
2. Lifetime secrets use regex classification. Port 3000 Is Taken is literally pinned to port 3000 — no other port counts.
3. Multi_condition secrets evaluate session aggregates. One Character Fix needs errors *and* a tiny patch in the same session.
4. Legend is a bulk Olympian check. It scans all achievements after a full rescan. The cost is negligible even on old hardware.
5. Caching v0.3.0+ means the 75,000-error regex loop only runs once. Subsequent scans skip unchanged sessions.

5 Questions to Check Your Understanding:

1. You trigger a port conflict on port 8080. Which line in the scanner decides whether Port 3000 Is Taken increments, and what is the result?
2. After discovering a secret badge, you delete `state.json` and force a rescan. The badge is visible again but the counter is lower. Why? Which file was the real source of truth?
3. One Character Fix requires `total_errors >= 4000`. If your lifetime error count is 3,999 and you make a tiny patch after 5 session errors, does the badge unlock? Which line is the gate?
4. Legend checks `total_achievements > 0` in Line 72. Why is this guard necessary, and what edge case does it prevent?
5. Your 2015 MacBook has 75,000 historical errors. The first full scan takes 2.3 seconds. After v0.3.0 caching, the second scan takes 80 ms. Which file stores the checkpoint that enables this speedup?